

Treinamento de IA



Boas-vindas

Querido(a) estudante,

Seja muito bem-vindo(a), ao curso Treinamento de IA, um curso que fará você viajar num mundo imaginário de teorias, ferramentas e práticas dos mais diferentes problemas que envolvem o ecossistema humano e, tudo isso, com o desenvolvimento da sua capacidade de raciocínio lógico, pensamento computacional, inteligência criativa, matemática aplicada e muita tecnologia.

Na primeira parte do curso, vamos entender o que é Aprendizado de Máquina - *Machine Learning* (ML), tipos de aprendizado e suas diferenças com a Inteligência Artificial (IA). Vamos conhecer os principais modelos de *Machine Learning* e como aplicá-los na prática de acordo com o modelo ideal. Além de saber como preparar os dados para a melhor aplicação.

Já na segunda parte do curso, você vai compreender as vantagens e desvantagens dos algoritmos, bem como, saber como e quando utilizar cada um dos algoritmos dos modelos de ML. Além disso, você vai aprender como avaliar e validar os modelos e entender algumas técnicas avançadas de modelagem.

Como entendemos que a IA tem por base os neurônios do cérebro humano, na terceira parte do curso, vamos entender um pouco sobre o treinamento de Redes Neurais - *Neural Networks* e Aprendizagem Profunda - *Deep Learning*. Além da arquitetura básica de uma rede neural, vamos entender como acontece o treinamento e a otimização por meio das redes neurais.

E, como nem tudo são caminhos simples, na quarta parte do curso, vamos treinar bastante com ferramentas e *frameworks* para *deep learning*. Vamos aprender a configurar ambientes, treinar e ajustar os hiperparâmetros em redes neurais.

Esperamos que você aproveite o máximo do curso, como parte dos saberes que te tornarão uma pessoa melhor para si, para quem você ama e para toda a sociedade, com capacidade de ajudar as pessoas a alcançarem melhores resultados em suas vidas e em suas profissões.

Todas as práticas são desenvolvidas utilizando a linguagem de programação [Python](https://www.python.org/)¹, no ambiente interativo [Collab](#)², todos os *Datasets* utilizados nas práticas são das bibliotecas importadas e estão nos códigos. Mas você também pode desenvolver a programação utilizando o [Visual Studio](#)³.

Junto com os materiais, você também tem acesso a todas as práticas e respostas num único arquivo:
Treinamento IA.ipynb

Vamos caminhar juntos nesta jornada? Estaremos com você!

¹ <https://www.python.org/>

² <https://colab.research.google.com/>

³ <https://code.visualstudio.com/>

PARTE I - FUNDAMENTOS DE *MACHINE LEARNING* (ML)

1. Introdução

Vamos começar explorando o que é *Machine Learning* (ML), como ele se diferencia de outras subáreas da Inteligência Artificial e onde ele é aplicado. Entender os conceitos básicos é fundamental para se construir um sólido entendimento sobre como treinar modelos de IA. E então, vamos nessa?

1.1 O que é *Machine Learning* e como ele se diferencia da Inteligência Artificial e do *Deep Learning*

Machine Learning é um campo da IA que está concentrada no desenvolvimento de algoritmos que são capazes de aprender padrões a partir de dados e fazer previsões ou decisões com base nesses padrões. Agora, imagine ensinar um computador a identificar diferentes tipos de frutas com base em suas cores e formatos. Você não precisa programar cada regra manualmente, como "se a fruta for amarela e alongada, é uma banana". Você só precisa treinar o modelo com um conjunto de dados para que ele aprenda sozinho a diferenciar cada fruta.

- **Inteligência Artificial (IA)** é o campo amplo que engloba as técnicas usadas para permitir que máquinas realizem tarefas de maneira "inteligente".
- ***Machine Learning* (ML)** é uma subárea da IA que utiliza algoritmos para aprender a partir de dados e fazer previsões ou decisões sem ser explicitamente programado.
- ***Deep Learning* (DL)** é uma subárea do ML que usa redes neurais profundas para processar dados complexos, como imagens e áudio, com um nível de precisão sem precedentes.

Curiosidade

Você sabia que a expressão "*Machine Learning*" foi criada pelo engenheiro estadunidense Arthur L. Samuel (1901 - 1990), em 1959? Ele definiu ML como um campo de estudo que "dá aos computadores a capacidade de aprender sem serem explicitamente programados".

1.2 Aplicações práticas de *Machine Learning* em diferentes setores

Machine Learning já faz parte do nosso dia a dia, há mais tempo do que você possa imaginar mesmo sem perceber, na maioria das vezes. Vamos dar uma olhada em algumas aplicações práticas?

- **Saúde:** Detecção precoce de doenças como câncer e diabetes, análise de exames e desenvolvimento de novos medicamentos.
- **Finanças:** Previsão de tendências de mercado, análise de risco de crédito e detecção de fraudes em transações bancárias.
- **Marketing:** Análise do comportamento do consumidor, recomendação de produtos e otimização de campanhas publicitárias.
- **Transporte:** Desenvolvimento de veículos autônomos, otimização de rotas e gerenciamento de frotas.

- **Educação:** Sistemas de recomendação de conteúdo, personalização de aprendizado e análise de desempenho acadêmico.

Esses são apenas alguns exemplos de como o *Machine Learning* tem se tornado, cada vez mais, um pilar essencial para inovação e eficiência em várias áreas de necessidades humanas.

1.3 Tipos de Aprendizado: Supervisionado, Não Supervisionado e por Reforço

No *Machine Learning*, existem três principais tipos de aprendizado:

- O **aprendizado supervisionado** onde o algoritmo é treinado com um conjunto de dados rotulados, de forma que cada exemplo de entrada está associado a uma saída desejada. É como ensinar um estudante a resolver equações fornecendo as respostas corretas para que ele aprenda as regras. Um exemplo é a previsão de preços de casas com base em características como número de quartos, localização e área.
- O **aprendizado não supervisionado** onde o algoritmo é treinado com um conjunto de dados sem rótulos, de forma que ele deve encontrar padrões ou grupos por conta própria, como organizar um conjunto de itens em categorias. Um exemplo é o agrupamento de clientes com base em comportamentos de compra, sem saber com antecedência quantos grupos ou categorias existem.
- O **aprendizado por reforço** onde o algoritmo aprende por meio da interação com o ambiente, recebendo recompensas ou penalidades conforme realiza ações. Ele é como um jogador que aprende as regras de um jogo por tentativa e erro. Um exemplo é o treinamento de um robô para jogar xadrez, onde cada movimento é avaliado em termos de vitória ou derrota.

Pense e Responda

Qual tipo de aprendizado você acredita que seria mais eficiente para treinar um sistema de recomendação de filmes, como o da Netflix? E para treinar um veículo autônomo?

Como Acontece na Prática

Que tal ver na prática a forma como acontece o *aprendizado por reforço* num exemplo aplicado a jogos? Então não deixe de assistir ao vídeo intitulado **“Inteligência Artificial Aprende a Jogar Super Mario Bros (Aprendizado por Reforço)”**, disponível no YouTube em <<https://www.youtube.com/watch?v=9nqwVYCxQIs>>

1.4 Atividade Prática: Classificação de imagens simples utilizando um algoritmo de *Machine Learning*

Vamos colocar a mão na massa? A atividade prática desta etapa é bem simples e vai te ajudar a aplicar os conceitos aprendidos. Usaremos a Linguagem [Python](#)¹ e a biblioteca [Scikit-Learn](#)² para classificar imagens de dígitos manuscritos.

Passo 1: Importe o **dataset digits** da biblioteca **Scikit-Learn**

```
from sklearn import datasets  
  
digits = datasets.load_digits()
```

Passo 2: Visualize os dados com [matplotlib](#)³

```
import matplotlib.pyplot as plt
plt.imshow(digits.images[0], cmap='gray')
plt.title(f'Label: {digits.target[0]}')
plt.show()
```

O resultado da visualização do código do Passo 2, gera a imagem como na Figura 1.

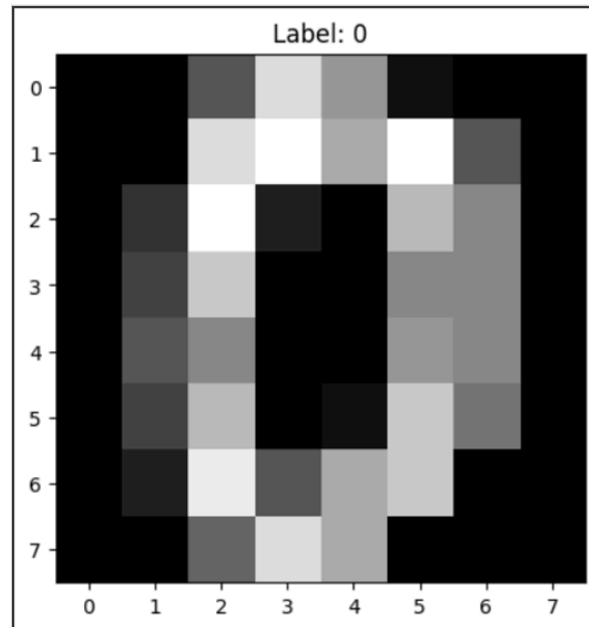


Figura 1

Passo 3: Treine um modelo de classificação utilizando as Máquinas de Vetores de Suporte - *Support Vector Machines* (SVM)

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC

X_train, X_test, y_train, y_test =
train_test_split(digits.data, digits.target, test_size=0.2, random_state=42)

model = SVC(gamma=0.001)
model.fit(X_train, y_train)
```

O resultado da execução do código do Passo 3, gera o resultado como na Figura 2.

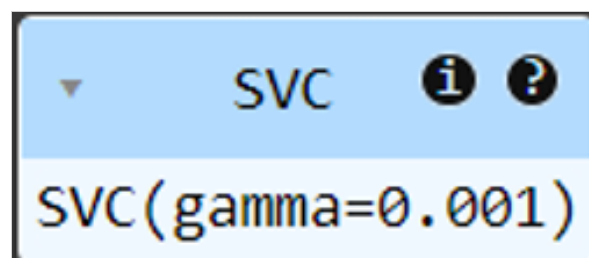


Figura 2

Passo 4: Avalie o desempenho do modelo

```
accuracy = model.score(X_test, y_test)
print(f'Acurácia do Modelo: {accuracy:.2f}')
```

O resultado da avaliação do código do Passo 4, gera o resultado como na Figura 3.

Figura 3

¹ <https://www.python.org/>

² <https://scikit-learn.org/>

³ <https://matplotlib.org/>

2. Principais Modelos em *Machine Learning*

Ao iniciar a jornada pelo mundo do *Machine Learning*, é importante entender os modelos que cada algoritmo pode executar. A escolha do modelo correto depende do problema a ser resolvido e do tipo de dados disponível. Nesta etapa, vamos estudar os principais modelos de *Machine Learning* e suas características, vamos também aplicá-las de forma prática e eficiente.

2.1 Modelos Comuns: Classificação, Regressão, *Clustering* e Redução de Dimensionalidade

Quando estudamos o campo do ML, existem quatro principais tipos de modelos que os algoritmos costumam realizar:

- a. A **classificação** que categoriza dados em grupos pré-definidos. Por exemplo, classificar e-mails como "spam" ou "não spam". Os algoritmos de classificação utilizam dados rotulados para aprender a identificar padrões e, dessa forma, prever a qual grupo uma nova observação pertence. Algoritmos de classificação são *Support Vector Machine* (SVM), Árvore de Decisão e *K-Nearest Neighbors* (KNN).
- b. A **regressão** é utilizada para prever números sequenciais, como a previsão de preços de imóveis ou a quantidade de vendas esperadas em uma loja. O objetivo é encontrar a relação entre as variáveis independentes e a variável dependente, ajustando a melhor linha ou curva que descreva os dados. Algoritmos de regressão são: regressão linear, regressão *ridge* e regressão *lasso*.
- c. O **clustering** agrupa critérios que têm características similares. O *clustering* trabalha com dados não rotulados e busca estruturar as informações de forma a identificar grupos, que chamamos de *clusters* dentro do *dataset*. Algoritmos de *clustering* são: *K-Means*, *DBSCAN* e *Hierarchical Clustering*.
- d. A **redução de dimensionalidade** é usada para simplificar conjuntos de dados, mantendo as informações mais relevantes. Reduzir a dimensionalidade facilita a visualização, o processamento dos dados e ajudar a melhorar a performance dos modelos. Algoritmos de Redução de Dimensionalidade são: *Principal Component Analysis* (PCA), *t-Distributed Stochastic Neighbor Embedding* (t-SNE) e *Linear Discriminant Analysis* (LDA).

Esses quatro modelos são fundamentais para resolver a maioria dos problemas enfrentados em *Machine Learning* e a sua escolha correta depende das características do problema e do tipo de dados.

2.2 Como escolher o modelo ideal para o seu problema

A escolha do modelo ideal para um problema depende das características dos dados e dos objetivos a serem alcançados. A primeira pergunta a ser feita é: *Qual é a natureza do problema?* Se for necessário prever uma categoria, como a espécie de uma planta com base em características físicas, o modelo ideal seria a *classificação*. Caso seja necessário prever um valor sequencial, como a temperatura média de uma cidade, o modelo ideal é a *regressão*.

- Os problemas de classificação podem ser utilizados quando há a necessidade de categorizar dados em grupos pré-definidos, como “aprovado” ou “reprovado”, “doente” ou “saudável”. É útil em diagnósticos médicos, classificação de texto e reconhecimento de imagem.
- Os problemas de Regressão podem ser utilizados para prever valores numéricos, como preços, idades e tempos de resposta. Sendo ideais para prever tendências e realizar análises financeiras.
- Os problemas de *clustering* podem ser utilizados quando se quer agrupar elementos que possuem características em comum e sem rótulos prévios. Aplicados em segmentação de clientes e análise exploratória de dados.
- Os problemas de redução de dimensionalidade podem ser utilizados quando há muitos atributos e é necessário simplificar o conjunto de dados sem perder informação relevante. Aplicados para visualização de dados complexos e melhoria da performance de modelos.

Para Aprender e Treinar

Experimente analisar um conjunto de dados que possua várias características (como o **dataset Iris** - conjunto de dados multivariado das três espécies de flores Iris: setosa, versicolor e virginica). Primeiro, utilize a tarefa de classificação para categorizar as espécies. Depois, aplique *clustering* para verificar se os grupos formados pelo algoritmo correspondem às classes reais do *dataset*.

2.3 Exemplos práticos de problemas reais resolvidos com cada tarefa

Os conceitos de classificação, regressão, *clustering* e redução de dimensionalidade fazem parte em inúmeros problemas do mundo real. Vamos ver alguns exemplos práticos:

- O algoritmo SVM pode ser utilizado para categorizar exames médicos como "positivo" ou "negativo" para uma determinada condição. Utilizando características como níveis de glicose e pressão arterial, é possível prever se um paciente possui diabetes.
- Os modelos de regressão linear podem ser utilizados para prever vendas futuras com base em variáveis como gasto em publicidade e tempo de operação. Isso ajuda empresas a planejarem estoques e gerirem cadeias de suprimentos.
- Os algoritmos de *clustering* como *K-means* são usados para segmentar clientes em grupos com base em características como comportamento de compra e demografia. Com isso, pode-se criar campanhas de marketing mais direcionadas.
- O PCA é utilizado para reduzir a complexidade de conjuntos de dados de imagens, eliminando redundâncias e facilitando a visualização das características mais importantes.

Curiosidade

Você sabia que o algoritmo *K-means* é bastante utilizado em estudos genéticos para agrupar organismos com base em características genéticas? Isso ajuda pesquisadores a entenderem melhor as relações entre diferentes espécies.

2.4 Atividade Prática: Implementação de um classificador binário em *Python*

Nesta atividade prática, vamos desenvolver um classificador binário utilizando a Linguagem *Python* e a biblioteca *Scikit-Learn*. O objetivo é aplicar os conceitos discutidos para implementar um modelo que diferencie duas classes de dados.

Passo 1: Use o *dataset breast_cancer* da biblioteca *Scikit-Learn* para classificar tumores como "malignos" ou "benignos"

```
from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

# Carregar o dataset

cancer = datasets.load_breast_cancer()
```

Passo 2: Divida o conjunto de dados em 80% para treinamento e 20% para teste

```
X_train, X_test, y_train, y_test =
train_test_split(cancer.data, cancer.target, test_size=0.2, random_state=42)
```

Passo 3: Treine um modelo SVM para realizar a classificação binária

```
model = SVC(kernel='linear')
model.fit(X_train, y_train)
```

O resultado do treinamento do código do Passo 3, gera a classificação como na Figura 4.

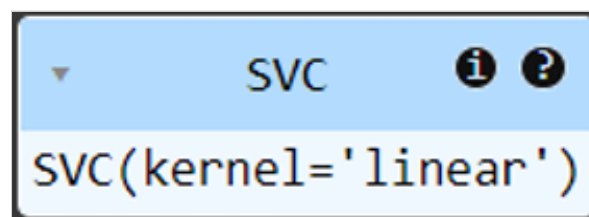


Figura 4

Passo 4: Calcule a acurácia do modelo no conjunto de teste

```
accuracy = model.score(X_test, y_test)
print(f'Acurácia do Modelo: {accuracy:.2f}')
```

O resultado do cálculo do código do Passo 4, gera o resultado como na Figura 5.

Acurácia do Modelo: 0.96

Figura 5

Pare e Pense

A acurácia é a melhor métrica para avaliar um modelo de classificação? Em cenários com classes desbalanceadas, como quando há muitos mais exemplos de "não spam" do que "spam", pode ser mais útil utilizar métricas como precisão e *recall*.

3. Preparação de Dados para *Machine Learning*

No processo de desenvolvimento de modelos de *Machine Learning*, uma das etapas mais importantes é a preparação dos dados. Dados de qualidade são fundamentais para que os modelos possam aprender padrões e realizar previsões precisas. Esta etapa envolve a coleta, limpeza, análise exploratória e transformação das informações. Nesta etapa vamos estudar a preparação de dados com técnicas essenciais para construir um *dataset* limpo e eficiente, sempre considerando o uso de ferramentas como o [Pandas](#)¹ e o *Scikit-Learn*.

3.1 Coleta e limpeza de dados

A coleta de dados é o primeiro passo para um projeto de *Machine Learning*. Os dados podem ser coletados de várias fontes, como bancos de dados relacionais, arquivos CSV, APIs e até *web scraping*. No entanto, os dados brutos geralmente apresentam inconsistências, como valores ausentes, duplicatas ou erros de digitação, que podem comprometer a qualidade do aprendizado.

Na coleta de dados, define-se as fontes confiáveis e relevantes para o problema. Utilize bibliotecas como ***pandas*** para ler arquivos CSV ou *requests* para acessar APIs e extrair dados.

Para a limpeza de dados, remova ou preencha valores ausentes, corrija erros tipográficos e elimine duplicatas. Utilize funções como ***dropna()*** e ***fillna()*** para tratar valores nulos e ***drop_duplicates()*** para remover dados repetidos.

Exemplo (supondo que existe um arquivo **dados.csv**):

```
import pandas as pd

# Coleta de dados de um CSV
data = pd.read_csv('dados.csv')

# Verificar valores ausentes
print(data.isnull().sum())

# Preencher valores ausentes com a média da coluna
data['idade'].fillna(data['idade'].mean(), inplace=True)

# Remover duplicatas
data.drop_duplicates(inplace=True)
```

Você sabia?

A qualidade dos dados pode impactar diretamente no desempenho do modelo. Em muitos casos, uma simples limpeza nos dados pode aumentar significativamente a precisão de um algoritmo.

3.2 Análise Exploratória de Dados e Visualização de Dados

A Análise Exploratória de Dados (EDA) é uma etapa importante para entender melhor o comportamento e a distribuição dos dados. Se você utiliza técnicas de EDA, pode identificar padrões, *outliers* e relações entre variáveis. O ***Pandas*** e o ***Matplotlib*** são bibliotecas essenciais para isso, permitindo criar gráficos que ajudam a visualizar os dados de maneira intuitiva.

A análise descritiva ajuda a verificar estatísticas básicas como média, mediana e desvio padrão usando o ***describe()*** para entender melhor as variáveis.

A visualização de dados pode utilizar gráficos como histogramas, *boxplots* e *scatter plots* para identificar a distribuição das variáveis e relações entre elas.

Exemplo (considerando a existência de data, carregado de dados.csv):

```
import matplotlib.pyplot as plt

# Estatísticas descritivas
print(data.describe())

# Histograma para visualizar a distribuição de uma variável
data['idade'].hist()
plt.xlabel('Idade')
plt.ylabel('Frequência')
plt.title('Distribuição de Idades')
plt.show()
```

Atenção

Outliers (valores atípicos) podem distorcer a média e o desvio padrão dos dados. Sempre visualize as distribuições para entender se a presença de *outliers* deve ser tratada, removida ou preservada.

3.3 Feature Engineering: extração, transformação e seleção de features

A etapa de *Feature Engineering* refere-se à criação de novas *features* (atributos) a partir dos dados brutos ou na transformação de *features* existentes para melhorar a capacidade de aprendizado do modelo. As *features* são importantes para o sucesso de um modelo de *Machine Learning*, pois representam informações importantes para a tomada de decisões.

A extração de *features* cria novas variáveis a partir das informações existentes. Por exemplo, a criação de uma variável "idade" a partir da data de nascimento.

A transformação de *features* aplica escalas ou normalizações para as variáveis com o intuito de garantir que estejam na mesma faixa de valores. Ferramentas como **StandardScaler** (transforma os dados para terem média zero e desvio padrão um) e **MinMaxScaler** (normaliza os dados para uma faixa específica, geralmente, entre 0 e 1) da biblioteca *Scikit-Learn* são úteis para essa tarefa.

A seleção de *features* utiliza técnicas como *SelectKBest* e *Recursive Feature Elimination* (RFE) para selecionar as features mais relevantes para o modelo. Isso reduz a dimensionalidade dos dados e melhora a performance do modelo.

Exemplo (considerando a existência de data, carregado de dados.csv)

```
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif

# Normalização dos dados
scaler = StandardScaler()
data[['idade', 'salario']] = scaler.fit_transform(data[['idade', 'salario']])

# Seleção de features mais relevantes
X = data.drop('target', axis=1)
y = data['target']
selector = SelectKBest(score_func=f_classif, k=5)
```

```
X_new = selector.fit_transform(X, y)

print(f'Features Selecionadas: {X.columns[selector.get_support()]})
```

Fato Curioso

Em alguns projetos, mais de 70% do tempo é dedicado para a preparação e engenharia de *features*, pois a performance do modelo depende da qualidade das variáveis fornecidas.

3.4 Atividade Prática: Limpeza e preparação de um *dataset* utilizando *Pandas* e *Scikit-Learn*

A atividade tem como objetivo realizar a limpeza e preparação de um *dataset* utilizando as técnicas apresentadas. O *dataset* utilizado será o “**Titanic**” (disponível no repositório de *datasets* da [Seaborn³](#)), que contém informações sobre passageiros e a sobrevivência ao naufrágio.

Passo 1: Coleta e leitura do *dataset*

```
import pandas as pd

import seaborn as sns

# Carregar o dataset Titanic

data = sns.load_dataset('titanic')
```

Passo 2: Limpeza de dados, removendo colunas irrelevantes e preenchendo valores ausentes em *age* com a média

```
data.drop(['deck', 'embark_town'], axis=1, inplace=True)

data['age'].fillna(data['age'].mean(), inplace=True)
```

Passo 3: Transformação de *features*, normalizando *age* e *fare* e codificando variáveis categóricas como *sex* e *embarked*

```
from sklearn.preprocessing import MinMaxScaler, LabelEncoder

scaler = MinMaxScaler()

data[['age', 'fare']] = scaler.fit_transform(data[['age', 'fare']])

le = LabelEncoder()

data['sex'] = le.fit_transform(data['sex'])

data['embarked'] = le.fit_transform(data['embarked'].astype(str))
```

Passo 4: Visualização de dados

```
import matplotlib.pyplot as plt

sns.histplot(data['age'], bins=10, kde=True)

plt.title('Distribuição de Idades dos Passageiros')

plt.show()
```

O resultado da visualização do código do Passo 4, gera o gráfico como na Figura 6.

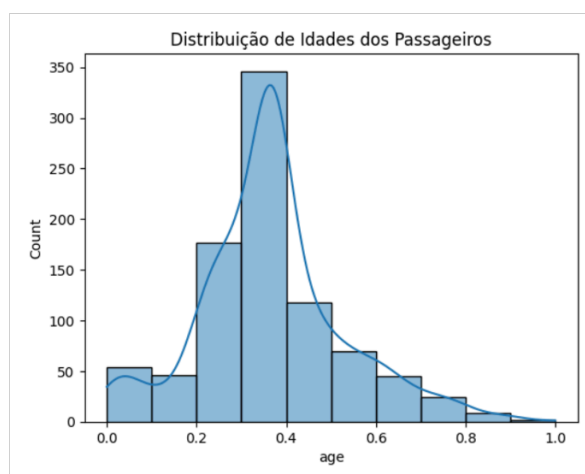


Figura 6

Saiba Mais

Outras transformações avançadas incluem binarização de variáveis numéricas, normalização logarítmica para lidar com distribuições enviesadas e *encoding* de variáveis categóricas com alta cardinalidade.

¹ <https://pandas.pydata.org/>

² <https://seaborn.pydata.org/>

PARTE II - MODELAGEM E TREINAMENTO DE MODELOS DE *MACHINE LEARNING*

4. Modelos Comuns de *Machine Learning*

O sucesso de um projeto de *Machine Learning* depende tanto dos dados e da preparação adequada, como também da escolha do modelo certo para o problema a ser resolvido. Cada algoritmo possui suas próprias características, vantagens e limitações.

Nesta etapa, vamos explorar os principais modelos de *Machine Learning*, suas aplicações, vantagens e desvantagens. Vamos também entender como decidir qual é o mais adequado para cada situação.

4.1 Algoritmos: Regressão Linear, Árvores de Decisão, SVM, *K-means*

Há uma variedade de algoritmos de *Machine Learning*, cada um desenvolvido para resolver um tipo específico de problema. Vamos estudar agora alguns dos modelos mais utilizados.

A Regressão Linear é um dos algoritmos mais simples e populares. Ele é utilizado para prever valores contínuos. O objetivo é encontrar a melhor linha reta que representa a relação entre as variáveis de entrada ou *features* e a variável de saída. Por exemplo, ela pode ser utilizada para prever o preço de um imóvel com base em variáveis como tamanho, número de quartos e localização.

- **Fórmula Geral:**

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Onde y é a variável dependente, ou seja, previsão, x_i são as variáveis independentes ou features e β_i são os coeficientes ou pesos ajustados pelo modelo.

As Árvores de Decisão funcionam como uma série de perguntas que dividem os dados em grupos menores e homogêneos. Cada nó da árvore representa uma decisão baseada em uma *feature* específica. A árvore continua se ramificando até que todos os pontos de dados sejam agrupados de acordo com uma classe ou valor.

- **Aplicações:**

Classificação de clientes para campanhas de *marketing*, decisão de crédito em instituições financeiras e diagnóstico médico.

O *Support Vector Machine* (SVM) é um algoritmo que busca encontrar o hiperplano que melhor separa as classes no espaço de características. Para problemas lineares, ele encontra a linha que maximiza a margem entre duas classes. Já para problemas não lineares, ele utiliza *kernels* para transformar os dados e encontrar um hiperplano em um espaço de maior dimensão.

- **Aplicações:** Reconhecimento de padrões em imagens, análise de sentimentos em texto e detecção de anomalias.

O *K-means* é um algoritmo de *clustering* que é utilizado para agrupar dados com base em suas características. Ele atribui cada ponto de dados a um dos k *clusters*. Isso minimiza a variabilidade dentro de cada *cluster*. O número k deve ser pré-definido e o algoritmo ajusta os centros dos *clusters* para otimizar a divisão.

- **Aplicações:** Segmentação de mercado, compressão de imagens e análise exploratória de dados.

O *K-means* é amplamente utilizado na análise exploratória de dados, onde o objetivo é identificar padrões ou agrupamentos naturais em grandes conjuntos de dados não rotulados. Ao dividir os dados em *clusters* com base em suas semelhanças, esse método ajuda a revelar padrões de comportamento de clientes ou grupos de usuários com características semelhantes, facilitando decisões baseadas em dados para estratégias de negócios ou análises científicas.

4.2 Quando e como utilizar cada modelo

A escolha do modelo depende de algumas considerações, como o tipo de variável alvo, que pode ser contínua ou categórica, a complexidade dos dados e a necessidade de interpretabilidade. Há algumas diretrizes gerais que podem ajudar na escolha do modelo.

Em relação à **regressão linear**, podemos considerar que:

- É um modelo que pode ser utilizado quando a variável de saída for contínua e houver uma relação linear aparente entre as variáveis de entrada e a saída.
- É ideal para prever preços, quantidades e outras variáveis numéricas.
 - Exemplos: revisão de temperatura, preços de imóveis e salários.

Em relação à **árvore de decisão**, podemos considerar que:

- É um modelo que pode ser utilizado quando for necessário tomar decisões baseadas em um conjunto de regras.
- Árvores de decisão são fáceis de interpretar e visualizar.
- São robustas a *outliers* e funcionam bem com dados categóricos e numéricos.
 - Exemplos: classificação de tipos de flores, avaliação de crédito e definição de categorias de clientes.

Em relação à **SVM**, podemos considerar que:

- É um modelo que pode ser utilizado quando o objetivo é separar classes complexas com margens bem definidas.
- Funciona bem com problemas de alta dimensionalidade e conjuntos de dados com poucas amostras.
 - Exemplos: classificação de imagens de gatos e cachorros, análise de sentimentos em textos e reconhecimento de escrita.

Em relação à ***K-means***, podemos considerar que:

- É um modelo que pode ser utilizado quando se deseja encontrar padrões ou grupos dentro dos dados, sem rótulos pré-definidos.
- Útil para segmentar clientes ou agrupar dados para análise exploratória.
 - Exemplos: segmentação de consumidores com base em comportamento de compra ou agrupamento de regiões geográficas por características populacionais.

Para Aprender e Treinar

Uma boa prática é experimentar diferentes modelos e avaliar seu desempenho com métricas como acurácia, precisão e erro médio. Utilizar validação cruzada e ajuste de hiperparâmetros ajuda a otimizar cada modelo para o problema específico.

4.3 Vantagens e desvantagens de cada abordagem

Cada modelo de *Machine Learning* possui suas vantagens e desvantagens, dependendo da natureza dos dados e do problema a ser resolvido.

Vamos pensar no modelo de ***regressão linear*** de ML e suas vantagens e desvantagens:

- Vantagens:
 - Simples de implementar e interpretar.
 - Funciona bem com conjuntos de dados pequenos e relações lineares.
- Desvantagens:
 - Não captura relações não lineares e é sensível a *outliers*.

Vamos pensar no modelo de ***árvore de decisão*** de ML e suas vantagens e desvantagens:

- Vantagens:
 - Fácil de entender, visualizar e interpretar.
 - Lida bem com variáveis categóricas e numéricas.
- Desvantagens:
 - Pode gerar modelos muito complexos e propensos a *overfitting* se não forem podadas adequadamente.

Vamos pensar no modelo de ***SVM*** de ML e suas vantagens e desvantagens:

- Vantagens:
 - Eficiente em espaços de alta dimensionalidade e bom para problemas de classificação complexos.
- Desvantagens:

- Sensível ao ajuste de hiperparâmetros e pode ser lento para grandes volumes de dados.

Vamos pensar no modelo de ***K-means*** de ML e suas vantagens e desvantagens:

- Vantagens:
 - Simples e fácil de implementar.
 - Rápido para conjuntos de dados grandes.
- Desvantagens:
 - O número de clusters k deve ser definido previamente e é sensível a pontos fora do padrão (*outliers*).

Pense e Responda

Qual modelo seria mais adequado para prever o valor de vendas de uma loja em um mês? E para identificar grupos de clientes com características semelhantes?

4.4 Atividade Prática: Treinamento de um modelo de regressão e um modelo de classificação

Nesta atividade, vamos treinar um modelo de Regressão Linear para prever valores contínuos e um modelo de Classificação utilizando uma Árvore de Decisão.

Passo 1: Utilizar um *dataset* simples para aplicar ambos os modelos

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris

# Dataset de regressão: geração de dados sintéticos
X_reg = pd.DataFrame({'tamanho': [50, 60, 70, 80, 90], 'idade': [10, 12, 15, 20, 25]})
y_reg = [300, 400, 500, 600, 700]

# Dataset de classificação: Iris
iris = load_iris()
X_class, y_class = iris.data, iris.target
```

Passo 2: Treinar um modelo de Regressão Linear para prever o preço de uma casa com base no tamanho e idade

```
regressor = LinearRegression()
regressor.fit(X_reg, y_reg)

pred = regressor.predict([[85, 18]]) # Previsão para uma casa de 85 m² e 18 anos
print(f'Previsão ')
```

Passo 3: Treinar um modelo de Árvore de Decisão para classificar as espécies de flores no *dataset* Iris

```
X_train, X_test, y_train, y_test = train_test_split(X_class, y_class, test_size=0.2, random_state=42)
```



```

classifier = DecisionTreeClassifier()
classifier.fit(X_train, y_train)
accuracy = classifier.score(X_test, y_test)
print(f'Acurácia no conjunto de teste: {accuracy:.2f}')

```

O resultado do treinamento do modelo do Passo 3, gera a acurácia como na Figura 7.

Acurácia no conjunto de teste: 1.00

Figura 7

Passo 4: Visualizar as previsões dos modelos e comparar com os valores reais.

O código abaixo plota a linha de regressão ajustada aos dados e destaca o ponto de previsão para um exemplo específico: casa com 85 m² e 18 anos.

```

import matplotlib.pyplot as plt

#plotar os pontos de dados e a linha de regressão
plt.scatter(X_reg['tamanho'], y_reg, color='blue', label='Dados Reais')
plt.plot(X_reg['tamanho'], regressor.predict(X_reg), color='red', label='Linha de Regressão')

# Destacar o ponto de previsão
plt.scatter(85, regressor.predict([[85, 18]]), color='green', s=100, label='Ponto de Previsão')

# Configurar o gráfico
plt.title('Regressão Linear - Previsão de Preço de Imóveis')
plt.xlabel('Tamanho (m²)')
plt.ylabel('Preço (mil)')
plt.legend()
plt.grid(True)
plt.show()

```

O resultado da primeira parte do código do Passo 4, gera o gráfico de regressão linear como na Figura 8.

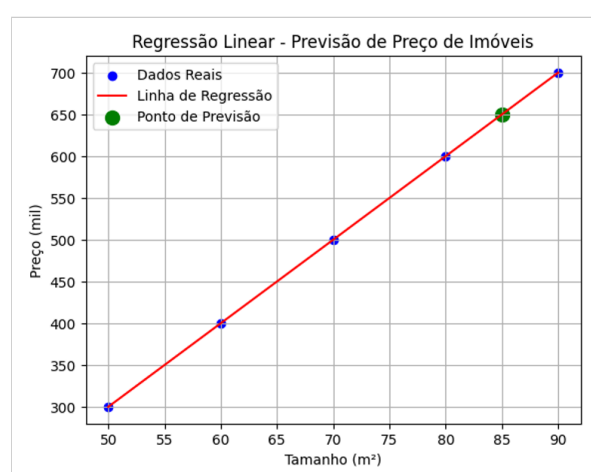


Figura 8

O código abaixo mostra um gráfico de dispersão das características do **dataset Iris** e como a Árvore de Decisão classifica cada ponto. Também calcula e imprime a matriz de confusão, que ajuda a avaliar a performance do modelo de classificação.


```

import seaborn as sns

from sklearn.metrics import confusion_matrix

import numpy as np

#Fazer previsões no conjunto de teste

y_pred = classifier.predict(X_test)

#Visualizar a matriz de confusão

conf_matrix = confusion_matrix(y_test, y_pred)

sns.heatmap(conf_matrix, annot=True, cmap='Blues', fmt='g')

plt.title('Matriz de Confusão - Classificação com Árvore de Decisão')

plt.xlabel('Previsão')

plt.ylabel('Valor Real')

plt.show()

#Visualizar as previsões de uma característica (exemplo: largura da pétala versus comprimento da pétala)

plt.scatter(X_test[:, 2], X_test[:, 3], c=y_pred, cmap='viridis', label='Previsão')

plt.scatter(X_test[:, 2], X_test[:, 3], c=y_test, edgecolor='red', label='Valor Real', marker='x')

#Configurar o gráfico

plt.xlabel('Largura da Pétala')

plt.ylabel('Comprimento da Pétala')

plt.title('Visualização das Previsões - Dataset Iris')

plt.legend()

plt.grid(True)

plt.show()

```

O resultado da segunda parte do código do Passo 4, gera o gráfico da matriz de confusão como na Figura 9. E, gera o gráfico da visualização das previsões como na Figura 10.

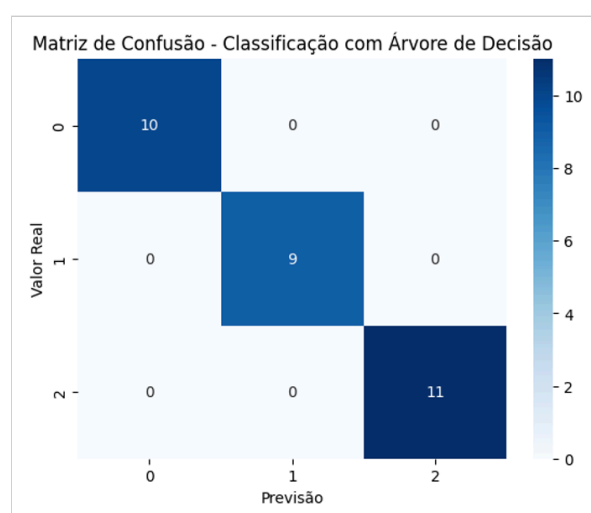
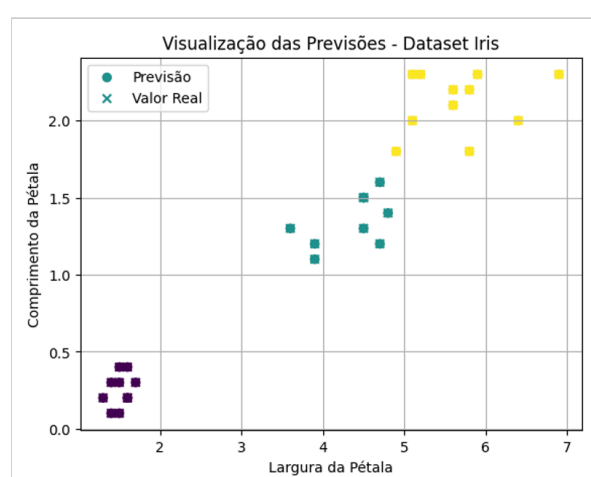


Figura 9



Saiba Mais

Experimente ajustar os hiperparâmetros dos modelos, como o *alpha* da Regressão Linear ou a profundidade máxima (*max_depth*) da Árvore de Decisão, para melhorar os resultados. Isso ajudará a entender o impacto de cada ajuste na performance dos modelos.

5. Avaliação e Validação de Modelos

Ao construir um modelo de ML, é importante você saber avaliar seu desempenho. Isso ajuda para que ele funcione corretamente e generalize bem para novos dados. A avaliação e a validação dos modelos ajudam a identificar falhas, como o *overfitting* e a ajustar os parâmetros para melhorar a performance.

Nesta etapa, vamos estudar as principais métricas de avaliação e técnicas de validação para buscar que o modelo desenvolvido seja robusto e confiável.

5.1 Métricas de avaliação: Acurácia, Precisão, Recall e F1-Score

As métricas de avaliação são usadas para medir o comportamento do modelo. A escolha da métrica certa depende do tipo de problema e do objetivo do modelo. Vamos entender as principais métricas de avaliação.

Se observarmos a acurácia ou *accuracy*, é uma métrica de avaliação que mede a proporção de previsões corretas em relação ao total de previsões. É uma métrica simples e fácil de interpretar, quando utilizada em problemas com classes desbalanceadas, por exemplo, 90% da classe A e 10% da classe B, a acurácia pode ser enganosa. Se o modelo sempre prever a classe A, ele terá 90% de acurácia, mas não será capaz de detectar a classe B.

Acurácia =
$$\frac{\text{Número de previsões corretas}}{\text{Número total de previsões}}$$

Se observarmos a precisão ou *precision*, é uma métrica que mede a proporção de previsões corretas para uma classe específica em relação a todas as previsões feitas para essa classe. É útil em cenários onde é importante minimizar falsos positivos.

Precisão =
$$\frac{\text{Verdadeiros Positivos (TP)}}{\text{Verdadeiros Positivos (TP)} + \text{Falsos Positivos (FP)}}$$

Se observarmos a sensibilidade ou *recall*, é uma métrica que mede a proporção de verdadeiros positivos em relação ao total de casos reais positivos. É útil em cenários onde se necessita minimizar falsos negativos.

Recall =
$$\frac{\text{Verdadeiros Positivos (TP)}}{\text{Verdadeiros Positivos (TP)} + \text{Falsos Negativos (FN)}}$$

Se observarmos o *F1-Score*, é uma métrica que considera a média harmônica entre a precisão e o *recall* e fornece um equilíbrio entre ambos. É uma métrica importante quando há um desbalanceamento entre classes e ambas as métricas precisam ser consideradas.

F1Score =
$$2 \cdot \frac{\text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}}$$

Curiosidade

Em problemas de classificação de spam, a precisão é importante para reduzir o número de e-mails legítimos marcados como spam (falsos positivos). O recall, por sua vez, é fundamental para garantir que a maioria dos spams seja detectada. A métrica *F1-Score* ajuda a equilibrar ambos os aspectos.

5.2 *Overfitting* e *Underfitting*: como detectar e corrigir

Durante o treinamento de um modelo, pode acontecer dele se ajustar muito bem aos dados de treinamento e, mesmo assim, apresentar um desempenho ruim em novos dados. Isso ocorre devido ao *overfitting*.

Por outro lado, quando o modelo não é capaz de capturar os padrões dos dados de treinamento, ele sofre de *underfitting*. Vamos entender como detectar e corrigir quando esses tipos de problemas acontecem.

O *overfitting* costuma ocorrer quando o modelo se ajusta muito bem aos dados de treinamento, porém pode falhar ao generalizar para novos dados. Em outras palavras, o modelo *memoriza* os dados de treinamento, em vez de aprender padrões generalizáveis.

É importante observar os sinais de ocorrência de *overfitting*, geralmente, na alta acurácia no treinamento e com baixa acurácia no teste.

Para corrigir a ocorrência do *overfitting*, é importante ficar atento:

- À regularização e quando se aplica as técnicas como *L1*, *L2* ou *Dropout* para simplificar o modelo.
- Ao aumento do conjunto de dados, quando se adiciona mais exemplos ou quando se aplica as técnicas de *data augmentation*.
- À simplificação do modelo quando se reduz o número de parâmetros ou camadas na rede neural.

O *Underfitting* costuma acontecer quando o modelo é muito simples para capturar os padrões dos dados de treinamento. Isso pode acontecer quando a capacidade do modelo ou o número de parâmetros é insuficiente para o problema.

É importante observar os sinais de *Underfitting*, que pode se dar pela baixa acurácia tanto no treinamento quanto no teste.

Para corrigir a ocorrência do *overfitting*, é importante ficar atento:

- Ao aumento da complexidade do modelo, quando adicionar mais camadas ou neurônios na rede neural.
- Ao treinamento por mais épocas, que pode aumentar o número de épocas para permitir que o modelo aprenda mais.
- Ao usar *features* mais relevantes, quando adicionar novas *features* que ajudem o modelo a capturar os padrões dos dados.

Para Aprender e Treinar

Testar o modelo em diferentes conjuntos de dados e ajustar seus hiperparâmetros para encontrar o equilíbrio entre *overfitting* e *underfitting*.

5.3 Técnicas de validação cruzada e *K-Fold*

A validação cruzada é uma técnica que pode ajudar a medir o desempenho do modelo de maneira mais robusta. Isso pode ser feito utilizando diferentes subconjuntos dos dados.

Em vez de dividir o conjunto de dados em treinamento e teste uma única vez, a validação cruzada divide o conjunto de dados várias vezes. Isso busca garantir que cada parte do conjunto seja utilizada como teste pelo menos uma vez.

A validação cruzada simples busca dividir os dados em um conjunto de treinamento e um conjunto de teste de forma aleatória. É uma técnica útil, mas pode não representar bem o desempenho do modelo em todos os cenários.

O *K-Fold Cross-Validation* busca dividir o conjunto de dados em k subconjuntos ou *folds*. O modelo é treinado k vezes, cada vez utilizando um subconjunto diferente como teste e os k-1 subconjuntos restantes como treinamento.

Como vantagem do *K-Fold Cross-Validation* podemos dizer que seja o uso de todo o conjunto de dados para treinamento e teste, que tem como resultando uma avaliação mais confiável. Além disso, a implementação na linguagem *Python*, como segue, considerando a existência dos parâmetros.

```
from sklearn.model_selection import cross_val_score

from sklearn.linear_model import LinearRegression


# Exemplo de validação cruzada com 5 folds

model = LinearRegression()

scores = cross_val_score(model, X_reg, y_reg, cv=5)

print(f'Médias das Acurácias com Validação Cruzada: {scores.mean():.2f}')
```

O *Leave-One-Out Cross-Validation* (LOOCV) é um caso extremo de *K-Fold*, onde k é igual ao número de amostras. Cada amostra é utilizada como teste uma vez, enquanto as demais são utilizadas como treinamento. Isso resulta em uma avaliação precisa, porém pode ser computacionalmente inviável por ser caro.

Dica

A validação cruzada é particularmente útil em conjuntos de dados pequenos, onde a divisão simples em treinamento e teste poderia resultar em variabilidade excessiva no desempenho do modelo.

5.4 Atividade Prática: Análise de métricas de desempenho para diferentes modelos treinados

Nesta atividade, vamos realizar uma comparação de diferentes modelos de *Machine Learning* utilizando métricas de avaliação e validação cruzada. O objetivo é analisar o comportamento dos modelos e determinar qual métrica é mais adequada para cada situação.

Passo 1: Utilizar o **dataset Iris** para classificação

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.tree import DecisionTreeClassifier

from sklearn.svm import SVC

from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score


#Carregar o dataset

iris = load_iris()

X, y = iris.data, iris.target


#Dividir os dados em treinamento e teste

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Passo 2: Treinar um classificador árvore de decisão e um SVM

```
#Modelo Decision Tree

dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

#Modelo SVM

svm_model = SVC()
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
```

Passo 3: Calcular precisão, *recall*, *F1-Score* e acurácia para ambos os modelos

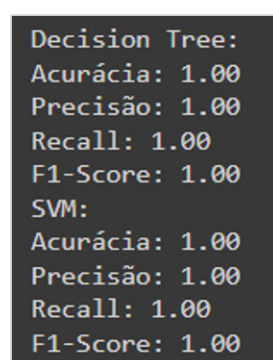
```
#Métricas para o Decision Tree

print("Decision Tree:")
print(f"Acurácia: {accuracy_score(y_test, y_pred_dt):.2f}")
print(f"Precisão: {precision_score(y_test, y_pred_dt, average='macro'):.2f}")
print(f"Recall: {recall_score(y_test, y_pred_dt, average='macro'):.2f}")
print(f"F1-Score: {f1_score(y_test, y_pred_dt, average='macro'):.2f}")

#Métricas para o SVM

print("SVM: ")
print(f"Acurácia: {accuracy_score(y_test, y_pred_svm):.2f}")
print(f"Precisão: {precision_score(y_test, y_pred_svm, average='macro'):.2f}")
print(f"Recall: {recall_score(y_test, y_pred_svm, average='macro'):.2f}")
print(f"F1-Score: {f1_score(y_test, y_pred_svm, average='macro'):.2f}")
```

O resultado dos cálculos para o *decision tree* e o SVM do Passo 3, gera o resultado como na Figura 11.



```
Decision Tree:
Acurácia: 1.00
Precisão: 1.00
Recall: 1.00
F1-Score: 1.00
SVM:
Acurácia: 1.00
Precisão: 1.00
Recall: 1.00
F1-Score: 1.00
```

Figura 11

Passo 4: Aplicar validação cruzada com 5 *folds* para ambos os modelos e comparar as acurácias

```
#Validação cruzada para decision tree

dt_cv_scores = cross_val_score(dt_model, X, y, cv=5)
print(f"Acurácia média com K-Fold (Decision Tree): {dt_cv_scores.mean():.2f}")

#validação cruzada para SVM

svm_cv_scores = cross_val_score(svm_model, X, y, cv=5)
print(f"Acurácia média com K-Fold (SVM): {svm_cv_scores.mean():.2f}")
```

O resultado das validações cruzadas do Passo 4, gera o resultado como na Figura 12.

Acurácia média com K-Fold (Decision Tree): 0.97
Acurácia média com K-Fold (SVM): 0.97

Figura 12

Passo 5: Após calcular as métricas e realizar a validação cruzada, é importante interpretar os resultados para saber qual é o modelo que está performando melhor e por quê.

- O modelo SVM teve desempenho superior em todas as métricas calculadas, indicando que ele consegue classificar as espécies de flores com mais precisão do que o *Decision Tree*.
- A acurácia média com validação cruzada também mostra que o SVM tem uma performance mais estável em diferentes subconjuntos dos dados.
- Se o problema real exigir mais precisão e *recall*, por exemplo, em um problema médico, o SVM seria a melhor escolha.
- No entanto, o *Decision Tree* é mais interpretável e pode ser mais adequado em cenários onde a explicabilidade é fundamental, por exemplo, decisões de crédito.

6. Técnicas Avançadas de Modelagem

Depois de explorar as principais abordagens de *Machine Learning*, chegou a hora de dar um passo adiante e mergulhar nas técnicas avançadas de modelagem. Por isso, nesta etapa, vamos apresentar práticas que permitem ajustar e melhorar o desempenho dos modelos, como regularização, seleção de hiperparâmetros e métodos de *ensemble learning*.

Essas estratégias ajudam a refinar os modelos e a extrair o máximo de desempenho a partir dos dados disponíveis. Vamos explorar cada uma delas?

6.1 Regularização e ajuste de parâmetros

Durante o treinamento de um modelo, o que se espera é encontrar os valores de parâmetros ou pesos, no caso de redes neurais, que melhor se ajustem aos dados. No entanto, modelos muito complexos podem sofrer de *overfitting*, aprendendo características irrelevantes e comprometendo a generalização para novos dados.

A regularização tem como objetivo ajudar a controlar a complexidade do modelo, penalizando pesos excessivos ou simplificando as relações entre as variáveis.

A *Regularização L1* ou *Lasso* é quando a regularização *L1* adiciona um termo de penalidade proporcional ao valor absoluto dos pesos ($|w|$) para a função de perda. Isso resulta em modelos mais esparsos, em que alguns coeficientes podem se tornar zero. Em outras palavras, *Lasso* realiza uma seleção de características, descartando variáveis irrelevantes.

$$\text{Função de Perda} = \text{Erro} + \lambda \sum |w|$$

Onde o λ é o coeficiente de regularização.

A *Regularização L2* ou *Ridge* é quando a regularização *L2* adiciona um termo de penalidade proporcional ao quadrado dos pesos (w^2) para a função de perda. Com isso, se incentiva o modelo a reduzir a magnitude dos pesos, sem eliminá-los. O que é útil quando todas as variáveis são importantes e os pesos precisam ser ajustados.

$$\text{Função de Perda} = \text{Erro} + \lambda \sum w^2$$

O *Elastic Net* combina as regularizações *L1* e *L2*. É uma boa opção quando se deseja aplicar a seleção de características (*L1*) e, ao mesmo tempo, manter a robustez do modelo (*L2*).

Vamos observar um exemplo prático na linguagem *Python*, considerando a existência dos parâmetros do modelo.


```

from sklearn.linear_model import Ridge, Lasso

#treinamento com Lasso (L1)

lasso_model = Lasso(alpha=0.1) #alpha é o parâmetro de regularização
lasso_model.fit(X_train, y_train)

#treinamento com Ridge (L2)

ridge_model = Ridge(alpha=0.1)
ridge_model.fit(X_train, y_train)

```

O resultado da execução do código anterior, gera o resultado como na Figura 13.

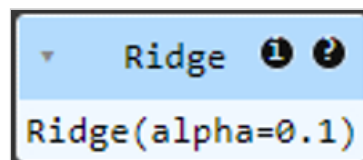


Figura 13

Atenção

A escolha do valor de λ ou α no *scikit-learn* é fundamental. Valores muito altos levam a um modelo excessivamente simples (*underfitting*), enquanto valores muito baixos não controlam suficientemente a complexidade (*overfitting*).

6.2 Seleção de hiperparâmetros: *Grid Search* e *Random Search*

Os hiperparâmetros são configurações que não podem ser ajustadas diretamente pelo modelo durante o treinamento, assim como o número de vizinhos no *KNN* ou a taxa de regularização em uma rede neural.

Encontrar a combinação ideal de hiperparâmetros pode melhorar significativamente o desempenho de um modelo. As técnicas mais comuns para essa seleção são o *Grid Search* e o *Random Search*.

O *Grid Search* é uma técnica que explora todas as combinações possíveis de hiperparâmetros dentro de um conjunto pré-definido. Esta técnica é útil para problemas onde o espaço de busca é pequeno e a execução é computacionalmente viável.

Vamos observar um exemplo prático na linguagem *Python*, considerando a existência dos parâmetros do modelo.

```

from sklearn.model_selection import GridSearchCV

#definir o modelo e os parâmetros a serem testados

parameters = {'alpha': [0.1, 0.5, 1.0, 10]}
ridge = Ridge()

#Aplicar Grid Search

grid_search = GridSearchCV(ridge, parameters, cv=5)
grid_search.fit(X_train, y_train)

print(f"Melhor parâmetro encontrado: {grid_search.best_params_}")

```


O resultado da execução do código anterior, gera o resultado como na Figura 14.

```
Melhor parâmetro encontrado: {'alpha': 1.0}
```

Figura 14

O *Random Search* é uma técnica que, em vez de explorar todas as combinações, amostra aleatoriamente diferentes combinações de hiperparâmetros. Esta técnica é mais eficiente que o *Grid Search* para problemas com muitos hiperparâmetros. Isso porque acaba testando apenas uma fração do espaço total.

Vamos observar um exemplo prático na linguagem *Python*, considerando a existência dos parâmetros do modelo.

```
from sklearn.model_selection import RandomizedSearchCV
import numpy as np

#definir o modelo e os parâmetros com valores aleatórios
parameters = {'alpha': np.random.uniform(0.01, 10, 100)}
lasso = Lasso()

#Aplicar Random Search
random_search = RandomizedSearchCV(lasso, parameters, n_iter=10, cv=5, random_state=42)
random_search.fit(X_train, y_train)

print(f"Melhor parâmetro encontrado: {random_search.best_params_}")
```

O resultado da execução do código anterior, gera o resultado como na Figura 15.

```
Melhor parâmetro encontrado: {'alpha': 0.38010329606603593}
```

Figura 15

Dica

Utilize *Random Search* quando o espaço de busca é grande e complexo. Caso contrário, o *Grid Search* é mais indicado, pois testa todas as combinações de maneira exaustiva.

6.3 Introdução ao *Ensemble Learning: Bagging, Boosting e Stacking*

O *Ensemble Learning* combina múltiplos modelos para obter um desempenho melhor do que qualquer modelo individual. A ideia é aproveitar diferentes algoritmos e suas forças para criar um modelo mais robusto e generalizável. Existem três estratégias principais para *ensemble learning* que vamos estudar, o *bagging*, o *boosting* e o *stacking*.

O ***Bagging***, um *bootstrap aggregatin*, é uma estratégia que treina vários modelos independentes em diferentes subconjuntos dos dados, gerados a partir de amostras com reposição ou *bootstrap*. Com isso, as previsões dos modelos são combinadas, o que pode ser por votação, em classificação, ou média, em regressão.

Um exemplo prático de *bagging* é o *Random Forest*, onde várias árvores de decisão são treinadas em subconjuntos dos dados.

E, a grande vantagem do *bagging*, é a redução da variância e a capacidade de ajudar a prevenir *overfitting*.

O **Boosting** é uma estratégia que treina modelos sequencialmente, onde cada novo modelo tenta corrigir os erros dos modelos anteriores. Podemos citar alguns modelos comuns de *boosting*, como *AdaBoost*, *Gradient Boosting* e *XGBoost*.

Um exemplo prático de *Boosting* é o *XGBoost*, que é amplamente utilizado em competições de *Machine Learning*, como as do *Kaggle*, por seu alto desempenho e capacidade de ajustar erros.

E, a grande vantagem do *Boosting* é que esta estratégia reduz o viés e melhora a performance geral do modelo. E, temos ainda o **Stacking**, uma estratégia que combina previsões de diferentes modelos, utilizando outro modelo, metamodelo, para aprender a melhor combinação. Por exemplo, um metamodelo de regressão linear pode combinar previsões de uma árvore de decisão e um SVM.

Vamos observar um exemplo prático de *Stacking* na linguagem *Python*, considerando a existência dos parâmetros do modelo.

```
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import Ridge
from sklearn.svm import SVR

#Definir modelos base e o metamodelo
estimators = [('ridge', Ridge()), ('svr', SVR())]
stack_model = StackingRegressor(estimators=estimators, final_estimator=Ridge())

#Treinar o modelo empilhado
stack_model.fit(X_train, y_train)
```

O resultado da execução do código anterior, gera o resultado como na Figura 16.

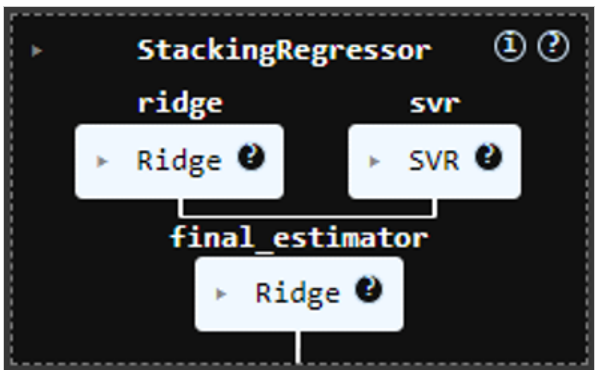


Figura 16

Para Aprender e Treinar

Aplique *Random Forest* em um conjunto de dados e compare com o desempenho de um único modelo de árvore de decisão. Observe como o ensemble melhora a estabilidade das previsões.

6.4 Atividade Prática: Implementação de *Grid Search* para ajuste de hiperparâmetros de um modelo

Agora é hora de aprender com uma atividade prática. Vamos aplicar o *Grid Search* para encontrar os melhores hiperparâmetros de um modelo de regressão. A ideia é ajustar o parâmetro *alpha* da regularização *Lasso* para obter o melhor desempenho.

Passo 1: Utilizar um *dataset* simples para treinar e testar a regularização *Lasso*

```
#importar as bibliotecas necessárias

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_absolute_error

#carregar o dataset California Housing

california = fetch_california_housing()

X, y = california.data, california.target

#dividir os dados em treinamento e teste

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Passo 2: Definir diferentes valores para o parâmetro *alpha* e aplicar o *Grid Search*

```
#definir o modelo e os parâmetros para grid search

parameters = {'alpha': [0.01, 0.1, 1.0, 10]}

lasso = Lasso()

#Aplicar Grid Search com validação cruzada de 4 folds

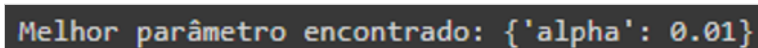
grid_search = GridSearchCV(lasso, parameters, cv=5)

grid_search.fit(X_train, y_train)

#Melhor parâmetro encontrado

print(f"Melhor parâmetro encontrado: {grid_search.best_params_}")
```

O resultado da execução do código do Passo 2, gera o resultado como na Figura 17.



```
Melhor parâmetro encontrado: {'alpha': 0.01}
```

Figura 17

Passo 3: Treinar o modelo Lasso com o melhor *alpha* e avaliar

```
#treinar o modelo com o melhor parâmetro

best_lasso = grid_search.best_estimator_

best_lasso.fit(X_train, y_train)

#fazer previsões no conjunto de teste

y_pred = best_lasso.predict(X_test)

#calcular e exibir o erro médio absoluto (MAE)

mae = mean_absolute_error(y_test, y_pred)

print(f"Erro Médio Absoluto (MAE) no Conjunto de Teste: {mae:.2f}")
```

O resultado do treinamento do modelo *Lasso* do Passo 3, gera o resultado do Erro Médio Absoluto / *Mean Absolute Error* - MAE, como na Figura 18.

Figura 18

Passo 4: Interpretação dos Resultados

Melhor Alpha: O valor de *alpha* encontrado pela *Grid Search* reflete o nível ótimo de regularização para o modelo. Se for um valor baixo, o modelo se ajustou melhor com pouca regularização. Se for um valor alto, a regularização foi importante para melhorar o desempenho.

MAE: O Erro Médio Absoluto indica a precisão do modelo no conjunto de teste. Um valor menor de MAE reflete previsões mais próximas dos valores reais. E, portanto, um modelo mais preciso.

Dica

Para problemas complexos e conjuntos de dados maiores, experimente aumentar o número de parâmetros ajustados no *Grid Search* ou aplique técnicas como *Random Search* para uma busca mais eficiente.

PARTE III - TREINAMENTO DE REDES NEURAIS E *DEEP LEARNING*

7. Introdução às Redes Neurais

Chegou a hora de explorar um conceito muito interessante do ML: as Redes Neurais. Muito se fala sobre como elas revolucionaram a Inteligência Artificial nos últimos anos, mas você já se perguntou o que realmente acontece dentro de uma rede neural? Para muitos, a ideia de *neurônios artificiais* pode parecer complexa, mas, na verdade, é possível entender esses conceitos de maneira clara e prática. Esta etapa vai te ajudar a desmistificar o funcionamento das redes neurais e como elas aprendem padrões a partir dos dados. Vamos começar?

7.1 Arquitetura básica de uma rede neural

As redes neurais são inspiradas no funcionamento do cérebro humano e tem o conceito de camadas de *neurônios* artificiais interconectados. A arquitetura básica de uma rede neural é composta por três tipos principais de camadas:

- A camada de entrada ou *Input Layer*, que é por onde os dados entram na rede. Cada neurônio representa uma *feature* ou características dos dados. Se, por exemplo, a rede for treinada para reconhecer dígitos manuscritos, a camada de entrada pode ter neurônios representando cada *pixel* de uma imagem.
- As camadas ocultas ou *Hidden Layers*, que realizam cálculos para aprender padrões nos dados. Quanto mais camadas ocultas, mais profunda e complexa é a rede. É nas camadas ocultas que a *magia* acontece, pois os pesos e *bias*¹ dos neurônios se ajustam para minimizar o erro da rede.
- A camada de saída é a camada final ou *Output Layer*, que gera a previsão ou classificação. Se o problema for classificar um dígito entre 0 e 9, a camada de saída terá 10 neurônios, cada um representando um dígito.

Cada neurônio na rede realiza uma operação matemática simples, multiplicando a entrada pelos pesos, soma um *bias*, que é um termo constante e passa o resultado por uma função de ativação. Esse processo é chamado de *forward propagation*. Em seguida, a rede ajusta os pesos para melhorar as previsões. Isso é feito por um processo chamado *backpropagation*, em que o erro é calculado e distribuído pela rede para otimizar os pesos.

Você sabia?

O termo *neurônio artificial* foi definido pelo neurocientista Warren McCulloch (1898 - 1969) e pelo matemático Walter Pitts (1923 - 1969), ambos estadunidenses que criaram o primeiro modelo matemático de um neurônio em 1943.

7.2 Perceptron: o primeiro neurônio artificial

O *Perceptron* é conhecido como a unidade básica de uma rede neural. Foi desenvolvido pelo psicólogo e notável pesquisador pioneiro das Redes Neurais Artificiais, o estadunidense Frank Rosenblatt (1928 - 1971) em 1958. Mesmo sendo muito simples, o *Perceptron* foi um marco na história da IA, pois introduziu a ideia de que máquinas poderiam aprender a partir de dados.

O funcionamento do *Perceptron* é bem parecido ao de um neurônio biológico. O *Perceptron* recebe múltiplas entradas, por exemplo, características como idade, peso, altura. Cada entrada possui um peso associado.

Depois acontece a combinação linear onde as entradas são multiplicadas pelos pesos e somadas, juntamente com o *bias*. E, por último, o resultado é passado por uma função de ativação, que decide se o *Perceptron* deve *ativar* ou disparar ou não, dependendo do valor da soma.

Se a soma das entradas ponderadas for maior que um determinado intervalo, o *Perceptron* ativa. Caso contrário, ele não ativa. Essa decisão é o que determina a saída final do *Perceptron*, 0 ou 1, por exemplo.

Um ponto importante é que o *Perceptron* só pode resolver problemas linearmente separáveis, como a classificação de dois grupos que podem ser separados por uma linha reta. Se as classes não puderem ser divididas por uma linha reta, como o problema do XOR, o *Perceptron* falha. Para superar essa limitação, surgiram redes neurais mais complexas, com várias camadas ocultas.

Pense e Responda

Se um *Perceptron* é capaz de distinguir entre dois grupos linearmente separáveis, o que aconteceria se as classes não puderem ser separadas por uma linha reta? Como resolver essa limitação?

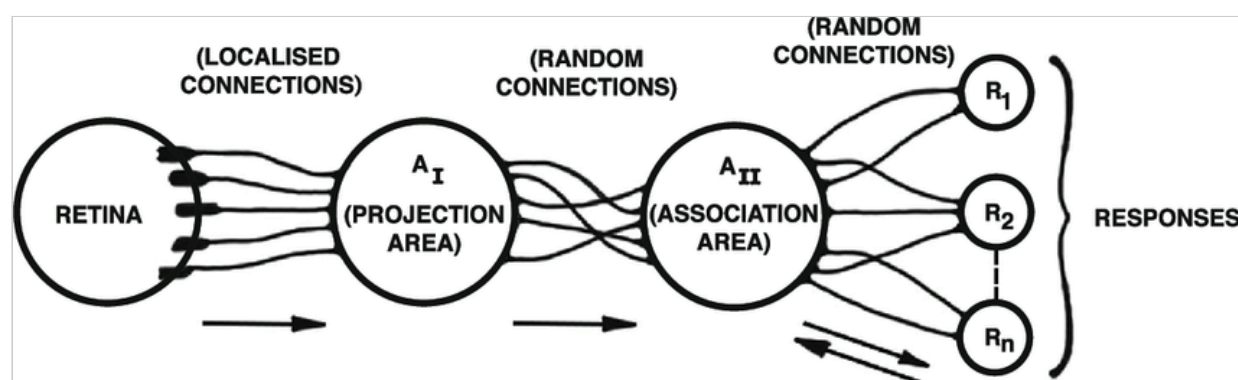


Imagem do conceito original do *Perceptron* de Frank Rosenblatt, na qual neurônios artificiais imitam a função do cérebro, transformando entradas na retina em respostas (Nash *et al.*, 2018 - *apud* Rosenblatt, 1958).

7.3 Funções de ativação: Sigmoid, ReLU, Tanh

As funções de ativação são responsáveis por introduzir *não-linearidade* em uma rede neural, permitindo que ela resolva problemas complexos. Sem essas funções, a rede se comportaria como uma combinação linear, limitada a resolver apenas problemas simples. As funções de ativação mais comuns são:

1. A função **Sigmoid** que transforma qualquer valor de entrada em uma saída entre 0 e 1. É usada para problemas de classificação binária.
 - Fórmula da função *Sigmoid*: $\sigma(x) = 1 / (1 + e^{(-x)})$.

- Desvantagem: A *Sigmoid* sofre do problema de gradiente desaparecendo ou *vanishing gradient*, o que dificulta o aprendizado em redes profundas.
2. A **ReLU (*Rectified Linear Unit*)** é uma função simples que retorna 0 para valores negativos e o seu próprio valor para valores positivos.
- Fórmula da *ReLU*: $f(x) = \max(0, x)$.
 - Vantagem: A *ReLU* é computacionalmente eficiente e reduz o problema de gradiente desaparecendo.
 - Desvantagem: Pode resultar no problema de *neurônios mortos*, em que alguns neurônios não ativam para nenhum dado de entrada.
3. A função **Tanh** é semelhante à *Sigmoid*, porém, mapeia valores para um intervalo entre -1 e 1, o que pode ajudar no aprendizado em redes mais profundas.
- Fórmula da função: $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$.
 - Vantagem: Produz saídas centradas em torno de zero, facilitando o aprendizado.
 - Desvantagem: Também sofre do problema de gradiente desaparecendo.

A escolha da função de ativação depende do problema e da arquitetura da rede neural. Redes profundas costumam usar a *ReLU* nas camadas ocultas e a *Sigmoid* ou *Softmax* utilizada principalmente na camada de saída de redes neurais.

Curiosidade

A *ReLU* se tornou popular na última década e é utilizada em quase todas as redes neurais profundas devido à sua simplicidade e eficácia em resolver problemas complexos.

7.4 Atividade Prática: Construção de uma rede neural simples para classificação de dados

Para aplicar os conceitos discutidos, vamos construir uma rede neural simples para classificar dígitos manuscritos usando a biblioteca *Keras* do [Tensorflow²](#) (biblioteca de código aberto que possui uma interface bastante intuitiva e que permite a criação, simulação e teste de modelos de ML e *Deep Learning*).

Passo 1: Importação das Bibliotecas e *Dataset*

```
import tensorflow as tf

from tensorflow.keras import layers, models

from sklearn.datasets import load_digits

from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt

# Carregar o dataset de dígitos

digits = load_digits()

X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.2,
random_state=42)
```

Passo 2: Criar uma rede neural com uma camada de entrada, duas camadas ocultas e uma camada de saída

```
model = models.Sequential()

model.add(layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)))

model.add(layers.Dense(32, activation='relu'))
```



```
model.add(layers.Dense(10, activation='softmax')) # 10 classes para dígitos de 0 a 9
```

Passo 3: Definir a função de perda, o otimizador e treinar a rede

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])  
model.fit(X_train, y_train, epochs=20, validation_data=(X_test, y_test))
```

O resultado da definição de perda, do otimizador e do treinamento da rede do Passo 3, gera o resultado como na Figura 19.

```
Epoch 1/20  
45/45 ————— 6s 46ms/step - accuracy: 0.2094 - loss: 4.0464 - val_accuracy: 0.5806 - val_loss: 1.2193  
Epoch 2/20  
45/45 ————— 1s 4ms/step - accuracy: 0.6523 - loss: 1.0405 - val_accuracy: 0.8194 - val_loss: 0.6314  
Epoch 3/20  
45/45 ————— 0s 8ms/step - accuracy: 0.8467 - loss: 0.5074 - val_accuracy: 0.8806 - val_loss: 0.3494  
Epoch 4/20  
45/45 ————— 0s 3ms/step - accuracy: 0.9122 - loss: 0.3118 - val_accuracy: 0.9222 - val_loss: 0.2730  
Epoch 5/20  
45/45 ————— 1s 7ms/step - accuracy: 0.9442 - loss: 0.1796 - val_accuracy: 0.9500 - val_loss: 0.2017  
Epoch 6/20  
45/45 ————— 0s 3ms/step - accuracy: 0.9648 - loss: 0.1404 - val_accuracy: 0.9528 - val_loss: 0.1695  
Epoch 7/20  
45/45 ————— 0s 3ms/step - accuracy: 0.9785 - loss: 0.1016 - val_accuracy: 0.9722 - val_loss: 0.1541  
Epoch 8/20  
45/45 ————— 1s 19ms/step - accuracy: 0.9902 - loss: 0.0686 - val_accuracy: 0.9694 - val_loss: 0.1464  
Epoch 9/20  
45/45 ————— 0s 2ms/step - accuracy: 0.9884 - loss: 0.0587 - val_accuracy: 0.9778 - val_loss: 0.1305  
Epoch 10/20  
45/45 ————— 0s 3ms/step - accuracy: 0.9956 - loss: 0.0416 - val_accuracy: 0.9806 - val_loss: 0.1200  
Epoch 11/20  
45/45 ————— 1s 8ms/step - accuracy: 0.9964 - loss: 0.0395 - val_accuracy: 0.9694 - val_loss: 0.1307  
Epoch 12/20  
45/45 ————— 0s 4ms/step - accuracy: 0.9946 - loss: 0.0385 - val_accuracy: 0.9750 - val_loss: 0.1208  
Epoch 13/20  
45/45 ————— 0s 5ms/step - accuracy: 0.9963 - loss: 0.0322 - val_accuracy: 0.9722 - val_loss: 0.1165  
Epoch 14/20  
45/45 ————— 2s 43ms/step - accuracy: 0.9960 - loss: 0.0307 - val_accuracy: 0.9778 - val_loss: 0.1125  
Epoch 15/20  
45/45 ————— 1s 4ms/step - accuracy: 0.9967 - loss: 0.0214 - val_accuracy: 0.9750 - val_loss: 0.1102  
Epoch 16/20  
45/45 ————— 0s 5ms/step - accuracy: 0.9976 - loss: 0.0216 - val_accuracy: 0.9694 - val_loss: 0.1140  
Epoch 17/20  
45/45 ————— 1s 28ms/step - accuracy: 0.9930 - loss: 0.0249 - val_accuracy: 0.9750 - val_loss: 0.1116  
Epoch 18/20  
45/45 ————— 0s 6ms/step - accuracy: 0.9977 - loss: 0.0161 - val_accuracy: 0.9667 - val_loss: 0.1214  
Epoch 19/20  
45/45 ————— 2s 38ms/step - accuracy: 0.9989 - loss: 0.0155 - val_accuracy: 0.9778 - val_loss: 0.1119  
Epoch 20/20  
45/45 ————— 1s 3ms/step - accuracy: 0.9997 - loss: 0.0106 - val_accuracy: 0.9778 - val_loss: 0.1090  
<keras.src.callbacks.history.History at 0x7be1bf682350>
```

Figura 19

Passo 4: Verificar a acurácia e visualizar algumas previsões

```
loss, accuracy = model.evaluate(X_test, y_test)  
print(f'Acurácia no conjunto de teste: {accuracy:.2f}')
```

O resultado da verificação da acurácia do Passo 4, gera o resultado como na Figura 20.

```
12/12 ————— 1s 119ms/step - accuracy: 0.9807 - loss: 0.0738  
Acurácia no conjunto de teste: 0.98
```

Figura 20

Saiba Mais

Experimente alterar o número de neurônios nas camadas ocultas e observe como isso afeta a performance da rede. A mudança de funções de ativação também pode impactar os resultados.

¹ *bias* (ou viés) é um termo adicional introduzido em cada neurônio, junto com os pesos, para ajudar a ajustar a saída da rede.

² <https://www.tensorflow.org/?hl=pt-br>

8. Treinamento e Otimização de Redes Neurais

Nesta etapa, vamos abordar um aspecto bastante importante do desenvolvimento de redes neurais, que é o processo de treinamento e otimização. O objetivo aqui é compreender como ajustar os parâmetros da rede para que ela aprenda de maneira eficaz e produza resultados precisos. Vamos aprender os principais algoritmos de otimização, as funções de perda e técnicas de regularização que ajudam a melhorar o desempenho dos modelos. Vamos explorar essas técnicas?

8.1 Algoritmos de otimização: *Gradient Descent*, *Stochastic Gradient Descent* e *Adam*

Os algoritmos de otimização são fundamentais no treinamento de redes neurais, pois determinam como os pesos dos neurônios serão ajustados para reduzir o erro e melhorar a precisão do modelo. O algoritmo mais básico e amplamente utilizado é o ***Gradient Descent*** ou Descida do Gradiente. É esse algoritmo que calcula o gradiente da função de perda em relação aos pesos e ajusta os pesos na direção oposta ao gradiente.

O *Gradient Descent* atualiza os pesos para a minimização da função de perda. A fórmula de atualização é dada por:

$$\omega = \omega - n \frac{\partial L}{\partial \omega}$$

Onde ω é o peso, n é a taxa de aprendizado e $\frac{\partial L}{\partial \omega}$ é o gradiente da perda em relação ao peso. Apesar de ser simples, o *Gradient Descent* precisa que todo o *dataset* seja processado para cada atualização, o que pode ser computacionalmente caro.

Em vez de calcular o gradiente sobre todo o conjunto de dados, o ***Stochastic Gradient Descent (SGD)***, atualiza os pesos usando um exemplo de cada vez, tornando o processo mais rápido e capaz de escapar de mínimos locais. A desvantagem é que as oscilações no caminho do gradiente podem tornar o SGD mais ruidoso e instável.

O ***Adaptive Moment Estimation (Adam)*** combina as vantagens do SGD com métodos que adaptam a taxa de aprendizado de acordo com a primeira e segunda ordem dos gradientes. O *Adam* ajusta os pesos com base no momento acumulado dos gradientes, e isso melhora a convergência e estabilidade do treinamento. A fórmula de atualização do *Adam* é mais complexa, mas, de maneira simplificada, ela equilibra as alterações no peso para cada parâmetro com base em como a rede está aprendendo.

Curiosidade

O *Adam* é amplamente utilizado em redes neurais profundas. Isso porque o Adam consegue manter um bom equilíbrio entre estabilidade e velocidade de convergência, tornando-o um dos otimizadores padrão na prática.

8.2 Funções de perda: MSE e *Cross-Entropy*

As funções de perda, que também são conhecidas como funções de custo, são utilizadas para medir o quão bem o modelo está se saindo. Elas calculam a diferença entre as previsões do modelo e os valores reais. Esse valor é minimizado durante o treinamento. A escolha da função de perda depende do tipo de problema a ser resolvido.

A **Mean Squared Error (MSE)** é uma função de perda bastante utilizada em problemas de regressão. Ela calcula o quadrado da diferença entre as previsões e os valores reais. Ela tira a média dessas diferenças. Quanto menor o valor de MSE, significa que melhor o modelo está se ajustando aos dados.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Onde y_i é o valor real e $\hat{y}_i$ é o valor previsto.

A **Cross-Entropy** é uma função de perda bastante utilizada em problemas de classificação. Ela mede a diferença entre a distribuição das previsões e a distribuição dos rótulos reais. A *Cross-Entropy* é particularmente útil quando há múltiplas classes e ajuda a melhorar a precisão do modelo em classificação.

$$L = - \sum_{i=1}^C y_i \log(p_i)$$

Onde C é o número de classes, y_i é a classe real e p_i é a probabilidade prevista para a classe i .

Pense e Responda

Em um problema de classificação binária, como prever se um cliente irá ou não realizar uma compra, qual função de perda seria mais adequada? E se fosse um problema de regressão, como prever o valor do imóvel com base em suas características?

8.3 Regularização: **Dropout** e **Batch Normalization**

Durante o treinamento de redes neurais é comum que o modelo apresente *overfitting*, ou seja, ele se ajusta tão bem aos dados de treinamento que não consegue generalizar para novos dados. Para evitar isso, podemos aplicar técnicas de regularização. Isso ajuda a manter o modelo mais simples e robusto.

O **Dropout** é uma técnica que *desliga*, de forma aleatória alguns neurônios durante o treinamento. Isso evita que o modelo se torne excessivamente dependente de determinadas conexões. Por exemplo, se a probabilidade de *dropout* for 0,5, metade dos neurônios será desativada em cada iteração. Isso força a rede a aprender características mais generalizáveis.

Implementação em *Keras* (considerando a existência de **model**):

```
from tensorflow.keras.layers import Dropout

model.add(Dropout(0.5)) # 50% de dropout na camada atual
```

O **Batch Normalization** busca normalizar as ativações de uma camada, ajustando a média e o desvio padrão das saídas. Isso acelera o treinamento e melhora a estabilidade da rede. Também evita que os gradientes explodam ou desapareçam.

Implementação em *Keras* (considerando a existência de **model** - "o objeto que representa a arquitetura da rede neural"):

```
from tensorflow.keras.layers import BatchNormalization

model.add(BatchNormalization())
```

Atenção

Dropout e ***Batch Normalization*** geralmente são aplicados apenas durante o treinamento. Na fase de teste ou inferência, esses mecanismos são desativados para garantir a performance ideal.

8.4 Atividade Prática: Treinamento de uma rede neural usando *Adam* e análise dos resultados

Para praticar os conceitos que estudamos nesta etapa, vamos treinar uma rede neural para classificar dígitos manuscritos utilizando o otimizador *Adam*. A atividade envolve o ajuste da função de perda, uso de *Dropout* para regularização e visualização dos resultados.

Passo 1: Importação das Bibliotecas e *Dataset*

```
import tensorflow as tf

from tensorflow.keras import layers, models

from sklearn.datasets import load_digits

from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt

# Carregar o dataset de dígitos

digits = load_digits()

X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.2,
random_state=42)
```

Passo 2: Criar uma rede neural com camadas densas *Dropout* e *Batch Normalization*

```
model = models.Sequential()

model.add(layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)))

model.add(layers.BatchNormalization())

model.add(layers.Dropout(0.5))

model.add(layers.Dense(32, activation='relu'))

model.add(layers.Dense(10, activation='softmax')) # 10 classes para dígitos de 0 a 9
```

Passo 3: Utilizar a função de perda *categorical_crossentropy* e o otimizador *Adam*

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=
['accuracy'])

history = model.fit(X_train, y_train, epochs=20, validation_data=(X_test, y_test))
```

O resultado da função de perda e o otimizador do código do Passo 3, gera o resultado como na Figura 21.

```

Epoch 1/20
45/45 ————— 3s 20ms/step - accuracy: 0.1921 - loss: 2.4816 - val_accuracy: 0.3528 - val_loss: 2.0172
Epoch 2/20
45/45 ————— 1s 10ms/step - accuracy: 0.5262 - loss: 1.4792 - val_accuracy: 0.7472 - val_loss: 0.9413
Epoch 3/20
45/45 ————— 0s 3ms/step - accuracy: 0.6561 - loss: 1.0939 - val_accuracy: 0.8556 - val_loss: 0.5646
Epoch 4/20
45/45 ————— 0s 3ms/step - accuracy: 0.7582 - loss: 0.8028 - val_accuracy: 0.9083 - val_loss: 0.4027
Epoch 5/20
45/45 ————— 0s 3ms/step - accuracy: 0.8211 - loss: 0.6545 - val_accuracy: 0.9194 - val_loss: 0.3146
Epoch 6/20
45/45 ————— 1s 19ms/step - accuracy: 0.8454 - loss: 0.5688 - val_accuracy: 0.9333 - val_loss: 0.2544
Epoch 7/20
45/45 ————— 0s 3ms/step - accuracy: 0.8629 - loss: 0.4646 - val_accuracy: 0.9389 - val_loss: 0.2150
Epoch 8/20
45/45 ————— 0s 3ms/step - accuracy: 0.8860 - loss: 0.4097 - val_accuracy: 0.9444 - val_loss: 0.1944
Epoch 9/20
45/45 ————— 0s 3ms/step - accuracy: 0.8914 - loss: 0.3413 - val_accuracy: 0.9472 - val_loss: 0.1760
Epoch 10/20
45/45 ————— 0s 4ms/step - accuracy: 0.9130 - loss: 0.3211 - val_accuracy: 0.9556 - val_loss: 0.1565
Epoch 11/20
45/45 ————— 0s 3ms/step - accuracy: 0.9029 - loss: 0.3170 - val_accuracy: 0.9611 - val_loss: 0.1518
Epoch 12/20
45/45 ————— 1s 23ms/step - accuracy: 0.9167 - loss: 0.2826 - val_accuracy: 0.9639 - val_loss: 0.1373
Epoch 13/20
45/45 ————— 1s 15ms/step - accuracy: 0.9162 - loss: 0.2596 - val_accuracy: 0.9639 - val_loss: 0.1384
Epoch 14/20
45/45 ————— 0s 3ms/step - accuracy: 0.9286 - loss: 0.2328 - val_accuracy: 0.9639 - val_loss: 0.1272
Epoch 15/20
45/45 ————— 0s 3ms/step - accuracy: 0.9329 - loss: 0.2170 - val_accuracy: 0.9694 - val_loss: 0.1249
Epoch 16/20
45/45 ————— 0s 4ms/step - accuracy: 0.9370 - loss: 0.2046 - val_accuracy: 0.9750 - val_loss: 0.1259
Epoch 17/20
45/45 ————— 0s 3ms/step - accuracy: 0.9405 - loss: 0.1841 - val_accuracy: 0.9722 - val_loss: 0.1268
Epoch 18/20
45/45 ————— 1s 23ms/step - accuracy: 0.9415 - loss: 0.1940 - val_accuracy: 0.9694 - val_loss: 0.1271
Epoch 19/20
45/45 ————— 2s 31ms/step - accuracy: 0.9453 - loss: 0.1804 - val_accuracy: 0.9694 - val_loss: 0.1301
Epoch 20/20
45/45 ————— 2s 19ms/step - accuracy: 0.9511 - loss: 0.1672 - val_accuracy: 0.9694 - val_loss: 0.1281

```

Figura 21

Passo 4: Plotar a acurácia e a perda durante o treinamento e a validação

```

plt.plot(history.history['accuracy'], label='Acurácia Treinamento')
plt.plot(history.history['val_accuracy'], label='Acurácia Validação')
plt.xlabel('Épocas')
plt.ylabel('Acurácia')
plt.legend()
plt.title('Acurácia do Modelo durante o Treinamento e Validação')
plt.show()

```

O resultado para plotar a acurácia e perda do Passo 3, gera o resultado como na Figura 22.

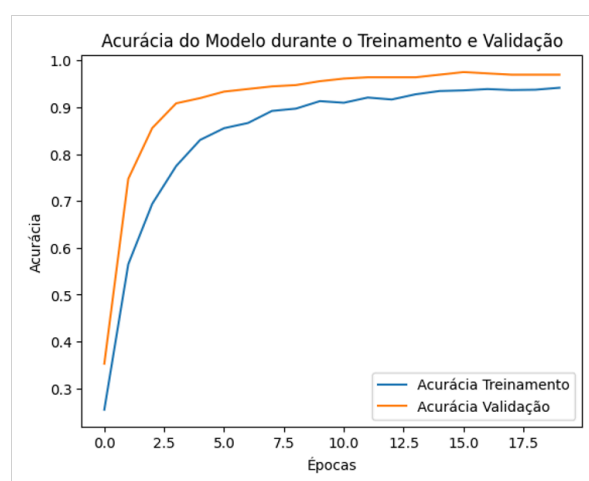


Figura 22

Para Aprender e Treinar

Experimente modificar a taxa de *dropout* e a função de ativação das camadas ocultas. Observe como essas alterações impactam a estabilidade e a performance da rede. Essa prática ajudará a entender como cada componente contribui para o desempenho final do modelo.

9. Estruturas Profundas: CNNs e RNNs

Até agora, aprendemos como construir redes neurais básicas e como otimizar seu treinamento. É importante saber que alguns problemas exigem redes mais especializadas para lidar com dados complexos, como imagens e séries temporais.

Redes Convolucionais / *Convolutional Neural Networks* (CNNs) e Redes Recorrentes / *Recurrent Neural Networks* (RNNs) são exemplos de estruturas profundas que se destacam pela sua capacidade de capturar padrões e dependências em dados que apresentam relações espaciais ou temporais. Vamos explorar as características dessas arquiteturas e entender quando e como utilizá-las.

9.1 Redes Convolucionais: estrutura, aplicações e treinamento

As Redes Convolucionais (CNN) são utilizadas para problemas que envolvem imagens e dados visuais. Sua arquitetura é composta por camadas convolucionais que filtram a entrada para extrair características. Estamos falando das bordas, texturas e padrões complexos. Essas redes foram inspiradas na organização do córtex visual humano e são capazes de reconhecer objetos em imagens com alta precisão.

A camada convolucional ou *convolutional layer* aplica um conjunto de filtros, também conhecidos por *kernels*, para a imagem de entrada, gerando mapas de ativação. Cada filtro é responsável por capturar uma característica específica, como bordas horizontais ou texturas.

A camada de *pooling* ou *pooling layer* busca reduzir a dimensionalidade dos mapas de ativação, mantendo as características mais importantes e eliminando informações redundantes. As operações de *max pooling* e *average pooling* são frequentemente utilizadas para resumir as ativações, o que resulta em uma rede mais eficiente e menos propensa ao *overfitting*.

Na camada de *flatten* nas camadas densas, ou *dense layers*, após as camadas convolucionais e de *pooling*, as ativações são achatadas ou *flattened* e passadas para camadas totalmente conectadas, onde a classificação ou regressão é realizada.

Há algumas aplicações interessantes das CNN:

- Na classificação de imagens, na identificação de objetos, reconhecimento facial e categorização de imagens.
- Na segmentação de imagens, na delimitação de áreas específicas em uma imagem, como células em exames médicos.
- No reconhecimento de padrões, na detecção de padrões em imagens, como bordas e texturas em cenas complexas.

Para exemplificar de forma prática, as CNN são amplamente utilizadas em sistemas de reconhecimento de placas de trânsito em veículos autônomos, onde cada placa deve ser identificada e classificada rapidamente.

Curiosidade

A arquitetura *LeNet*, desenvolvida pelo cientista da computação franco-americano Yann LeCun em conjunto com pesquisadores da AT&T Bell Labs <<https://www.bell-labs.com/>> em 1989, sendo uma das primeiras CNN aplicadas para reconhecimento de dígitos manuscritos e deu origem às modernas arquiteturas como: *LeNet-5*, *AlexNet*, *VGGNet* e *ResNet*.

9.2 Redes Recorrentes: arquitetura e aplicação em séries temporais

As Redes Recorrentes (RNN) são projetadas para considerar dados sequenciais, como séries temporais, texto e áudio. A principal característica das RNN é a capacidade de manter um estado interno que *lembra* informações sobre entradas anteriores, permitindo que a rede capture dependências temporais e contextuais.

Na arquitetura básica das RNN, cada neurônio em uma RNN recebe a entrada atual, a saída dos neurônios da camada anterior e a própria saída da iteração anterior. Isso cria um ciclo que conecta os neurônios de maneira temporal, permitindo que a rede *lembre* informações passadas.

Como Acontece na Prática

Quer ver como uma Rede Neural Recorrente (RNN) funciona na prática? Então, não deixe de assistir, procure de fato entender o processo e execute o código que está colocado junto do vídeo intitulado “***MariFlow - Self-Driving Mario Kart w/Recurrent Neural Network***” ou algo como: *Mario Kart Autônomo com Rede Neural Recorrente*”, disponível no YouTube em <https://www.youtube.com/watch?v=Ipi40cb_RsI>

No problema do gradiente desvanecente, o *vanishing gradient*, as RNN tradicionais sofrem do problema em que o gradiente diminui exponencialmente ao retropropagar o erro por meio do tempo. Isso dificulta o aprendizado de dependências longas.

Para buscar resolver esse problema, surgiram as **Redes de Memória de Curto e Longo Prazo / Long Short-Term Memory (LSTM)** e as **Unidades Recorrentes Gated / Gated Recurrent Units (GRU)**. Essas arquiteturas possuem mecanismos internos que buscam controlar quais informações devem ser *lembradas* ou *esquecidas* em cada iteração. Isso os torna capazes de aprender dependências temporais mais longas.

Há algumas aplicações interessantes das RNN:

Para a previsão de séries temporais, podemos ter aplicações na previsão de valores futuros com base em dados históricos, como preços de ações ou consumo de energia.

Para o **Processamento de Linguagem Natural (PLN) / Processing Natural Language (PNL)**, podemos ter aplicações na análise de sentimentos, tradução automática e geração de texto.

Para a análise de áudio, podemos ter aplicações no reconhecimento de fala, detecção de sons e síntese de voz.

Para exemplificar de forma prática, as RNN são utilizadas em assistentes virtuais como *Alexa* <<https://www.amazon.com/alexa>> e *Google Assistant* <<https://assistant.google.com/>>. Isso acontece para entender e responder comandos de voz, mantendo o contexto das interações anteriores.

Fato Curioso

As LSTM, criadas por Hochreiter e Schmidhuber em 1997, são até hoje uma das arquiteturas mais utilizadas em processamento de linguagem natural e séries temporais, graças à sua capacidade de capturar dependências de longo prazo.

9.3 Atividade Prática: Treinamento de uma CNN para classificação de imagens de dígitos

Vamos colocar em prática o aprendizado sobre Redes Convolucionais com um exemplo clássico que é a classificação de imagens de dígitos manuscritos utilizando uma CNN.

Nesta prática, o objetivo é construirmos, treinarmos e avaliarmos uma CNN usando o *dataset* **Modified National Institute of Standards and Technology database (MNIST)** o qual é uma grande coleção de dígitos manuscritos, que contém 70.000 imagens de dígitos (0 a 9) em escala de cinza.

Passo 1: O *dataset* MNIST pode ser facilmente carregado a partir da biblioteca *Keras*


```
import tensorflow as tf

from tensorflow.keras import layers, models

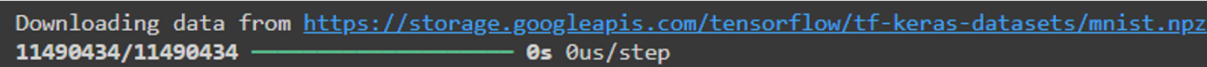
# Carregar o dataset MNIST

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalizar os dados

X_train, X_test = X_train / 255.0, X_test / 255.0 # Escalar valores entre 0 e 1
```

O resultado para importar o *dataset* do Passo 1, gera o resultado como na Figura 23.



```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
```

Figura 23

Passo 2: A CNN será composta por duas camadas convolucionais e camadas de *pooling*

```
model = models.Sequential()

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))

model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.add(layers.MaxPooling2D((2, 2)))

model.add(layers.Flatten())

model.add(layers.Dense(64, activation='relu'))

model.add(layers.Dense(10, activation='softmax')) # 10 classes para dígitos de 0 a 9
```

Passo 3: Compilar o modelo com o otimizador Adam e a função de perda *sparse_categorical_crossentropy*

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=
['accuracy'])

history = model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test))
```

O resultado da compilação do modelo e da função do Passo 3, gera o resultado como na Figura 24.



```
Epoch 1/10
1875/1875 ————— 108s 57ms/step - accuracy: 0.8995 - loss: 0.3377 - val_accuracy: 0.9847 - val_loss: 0.0474
Epoch 2/10
1875/1875 ————— 99s 53ms/step - accuracy: 0.9841 - loss: 0.0526 - val_accuracy: 0.9890 - val_loss: 0.0337
Epoch 3/10
1875/1875 ————— 134s 48ms/step - accuracy: 0.9889 - loss: 0.0359 - val_accuracy: 0.9899 - val_loss: 0.0294
Epoch 4/10
1875/1875 ————— 117s 35ms/step - accuracy: 0.9926 - loss: 0.0212 - val_accuracy: 0.9925 - val_loss: 0.0247
Epoch 5/10
1875/1875 ————— 79s 34ms/step - accuracy: 0.9950 - loss: 0.0160 - val_accuracy: 0.9875 - val_loss: 0.0400
Epoch 6/10
1875/1875 ————— 84s 35ms/step - accuracy: 0.9955 - loss: 0.0128 - val_accuracy: 0.9904 - val_loss: 0.0287
Epoch 7/10
1875/1875 ————— 82s 35ms/step - accuracy: 0.9968 - loss: 0.0098 - val_accuracy: 0.9909 - val_loss: 0.0311
Epoch 8/10
1875/1875 ————— 83s 35ms/step - accuracy: 0.9970 - loss: 0.0080 - val_accuracy: 0.9893 - val_loss: 0.0389
Epoch 9/10
1875/1875 ————— 81s 35ms/step - accuracy: 0.9975 - loss: 0.0069 - val_accuracy: 0.9851 - val_loss: 0.0616
Epoch 10/10
1875/1875 ————— 83s 35ms/step - accuracy: 0.9979 - loss: 0.0063 - val_accuracy: 0.9914 - val_loss: 0.0370
```

Figura 24

Passo 4: Verificar a acurácia no conjunto de teste

```
loss, accuracy = model.evaluate(X_test, y_test)
```



```
print(f'Acurácia no conjunto de teste: {accuracy:.2f}')
```

O resultado para verificar a acurácia no conjunto de teste do Passo 4, gera o resultado como na Figura 25.

```
313/313 ————— 7s 22ms/step - accuracy: 0.9885 - loss: 0.0494  
Acurácia no conjunto de teste: 0.99
```

Figura 25

Passo 5: Mostrar algumas imagens do conjunto de teste com as previsões da CNN

```
import matplotlib.pyplot as plt  
import numpy as np  
  
predictions = model.predict(X_test)  
for i in range(5):  
    plt.imshow(X_test[i], cmap='gray')  
    plt.title(f'Previsão: {np.argmax(predictions[i])}')  
    plt.show()
```

O resultado de mostrar algumas imagens do conjunto e testes do Passo 4, gera o resultado como na Figura 26.

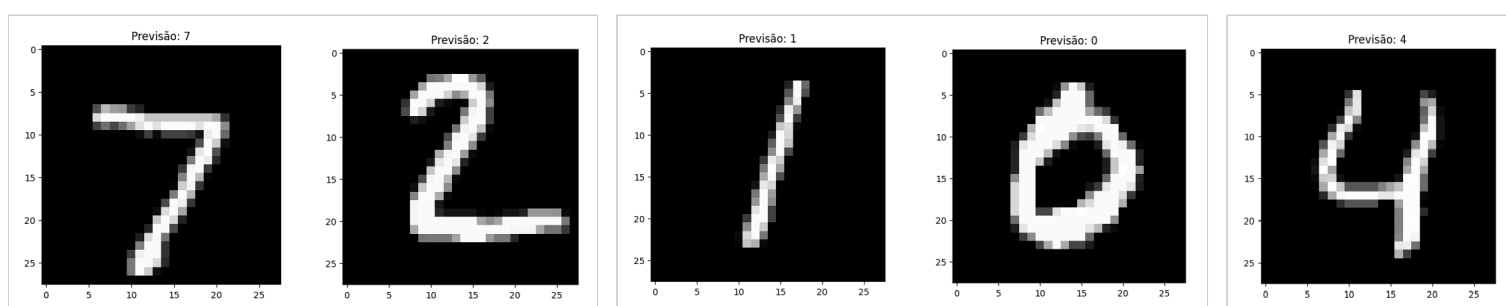


Figura 26 (a)

Figura 26 (b)

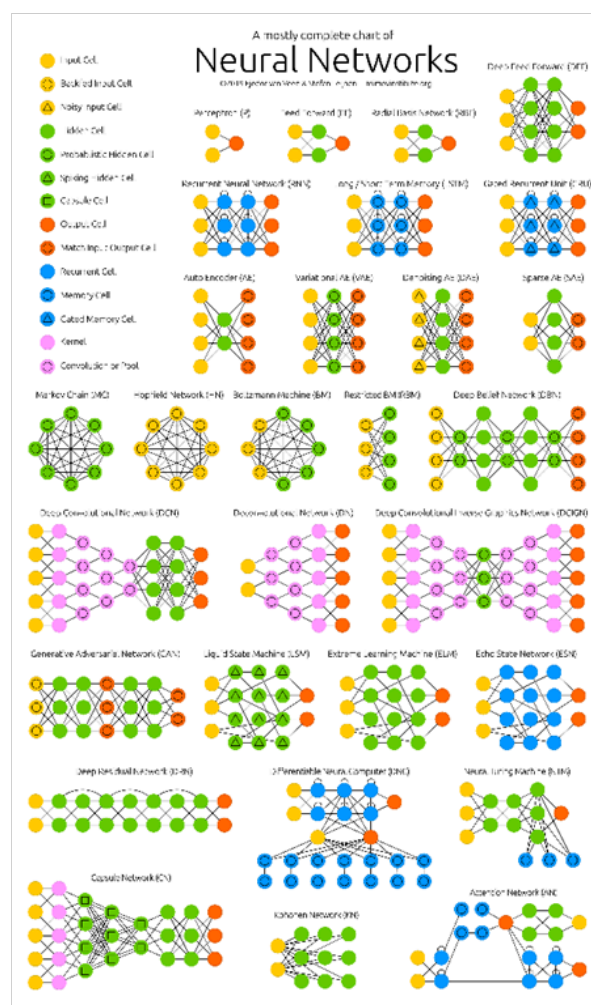
Figura 26 (c)

Dica

Experimente aumentar o número de filtros ou adicionar mais camadas convolucionais e observe como isso impacta a performance da CNN. Modifique o número de épocas e analise o comportamento do modelo em termos de *overfitting* e *underfitting*.

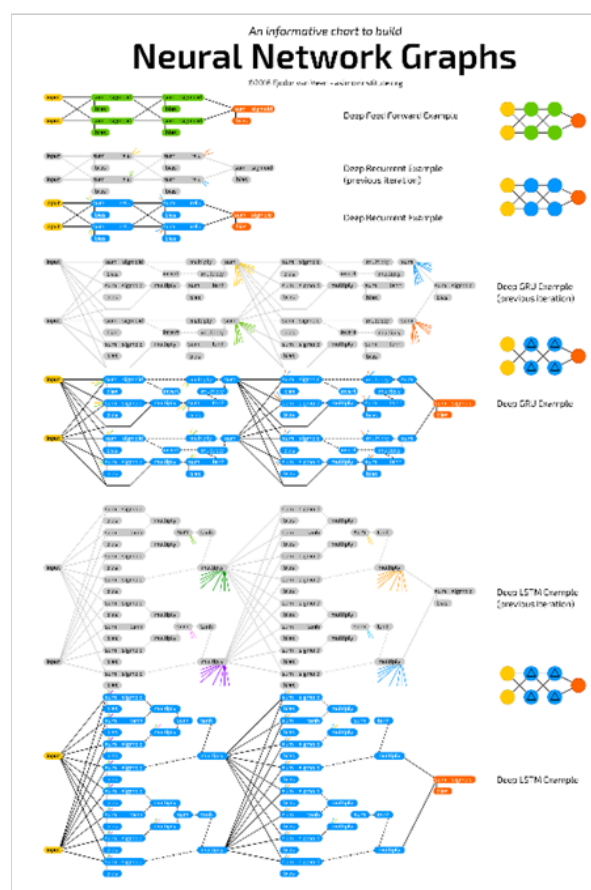
O conceito de redes neurais pode ser mais facilmente compreendido por meio de meios visuais detalhados de forma clara e direta. Nesse sentido indicamos dois excelentes trabalhos do pesquisador Fjodor van Venn do Instituto Asimov de pesquisas em IA (Países Baixos).

O primeiro trabalho traz um bastante completo mapa com as arquiteturas de diferentes redes neurais.



(Veen, 2016)

E o segundo, demonstrando vários modos de conectividade entre as camadas e diferentes tipos de células.



(Veen, 2017)

PARTE IV - FERRAMENTAS E *FRAMEWORKS* PARA *DEEP LEARNING*

10. Introdução ao *TensorFlow* e *PyTorch*

No mundo do *Deep Learning*, os *frameworks TensorFlow* e [PyTorch](#)¹ são os mais populares e bastante utilizados por desenvolvedores e pesquisadores. Os dois oferecem uma ampla variedade de ferramentas para construção, treinamento e avaliação de redes neurais profundas.

Por mais que existam semelhanças entre os dois, cada um possui características e sintaxes específicas que podem ser mais adequadas para determinados tipos de projetos. Nesta etapa, vamos estudar uma introdução a esses dois *frameworks*, suas diferenças e como construir um modelo simples em cada um deles. Vamos começar?

10.1 Configuração do ambiente e primeiros passos

Antes de construir modelos utilizando *TensorFlow* e *PyTorch* é importante saber configurar o ambiente de desenvolvimento. Ambos os *frameworks* podem ser instalados facilmente em um ambiente *Python* utilizando o *pip* (gerenciador de pacotes para a linguagem de programação *Python* - *Pip Install Packages*).

Para instalar o *framework TensorFlow*, existem algumas recomendações em sua criação, como um ambiente virtual para isolar as dependências do projeto. Na linguagem *Python*, temos:

```
pip install tensorflow
```

Se você quiser utilizar a versão com suporte para GPU, placas gráficas da NVIDIA, a instalação deve ser feita com o código:

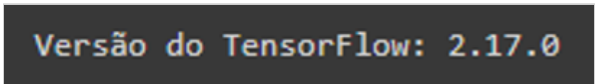
```
pip install tensorflow-gpu
```

Agora, vamos para os primeiros passos com o *TensorFlow*:

```
import tensorflow as tf

print(f"Versão do TensorFlow: {tf.__version__}")
```

O resultado do código anterior, gera o resultado como na Figura 27.



```
Versão do TensorFlow: 2.17.0
```

Figura 27

Agora, vamos instalar o *PyTorch*. A versão deve ser escolhida com base no sistema operacional, versão do *Python* e suporte para [CUDA](#)¹ (caso se for utilizar Unidades de Processamento Gráfico / *Graphics Processing Units* - GPU):

```
pip install torch torchvision
```

Para o suporte com a GPU:

```
pip install torch torchvision torchaudio --index-url
```

Muito bem, vamos seguir com os primeiros passos com o *PyTorch*:

```
import torch
print(f"Versão do PyTorch: {torch.__version__}")
```

O resultado do código anterior, gera o resultado como na Figura 28.

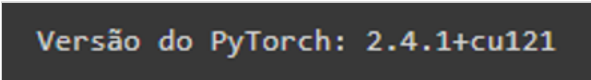


Figura 28

Dica

Para verificar se ambos os *frameworks* estão utilizando a GPU corretamente, utilize ***tf.test.is_gpu_available()*** no *TensorFlow* e ***torch.cuda.is_available()*** no *PyTorch*. Isso permite que a configuração seja realizada com sucesso.

10.2 Comparação entre *TensorFlow* e *PyTorch*

Apesar de os dois *frameworks* que estamos estudando serem usados para construir redes neurais profundas, existem algumas diferenças importantes que podem tornar um mais adequado do que o outro, dependendo do projeto.

Tabela 1: Diferenças importantes entre *TensorFlow* e *PyTorch*

Características	<i>TensorFlow</i>	<i>PyTorch</i>
Lançamento	2015, desenvolvido pelo Google	2016, desenvolvido pelo Facebook (Meta)
Paradigma	Definição estática de gráficos (TensorFlow 1.x) e dinâmica com tf.function (TensorFlow 2.x)	Definição dinâmica de gráficos (eager execution)
Curva de Aprendizado	Levemente mais íngreme devido à API complexa	Mais intuitivo e semelhante ao NumPy
Ecossistema	Suporte para TensorBoard (visualização) e TensorFlow Serving (deploy)	Integração facilitada com PyTorch Lightning
Documentação e Suporte	Documentação extensiva e tutoriais oficiais do Google	Comunidade ativa e suporte para pesquisa
Performance	Similar com suporte a GPUs e TPUs	Similar com suporte a GPUs

O *TensorFlow* é um *framework* ideal para aplicações que exigem escalabilidade, como *deploy* de modelos em produção. Para quem deseja aproveitar ferramentas como o *TensorBoard* (visualização) e o *TensorFlow Serving* (implementação de modelos), também é indicado. A biblioteca oferece ainda, suporte para desenvolvedores que desejam usar [Tensor Processing Units³](#) (TPU).

O *PyTorch* é um *framework* bastante utilizado por pesquisadores e para projetos de prototipagem rápida. Isso porque a sua sintaxe é mais próxima ao *Python* tradicional e integração com a [NumPy⁴](#). Sua definição dinâmica de gráficos torna o processo de depuração e modificação de modelos mais simples e intuitivo.

Para Aprender e Treinar

Escolha um dos *frameworks* e desenvolva uma rede neural simples, como a classificação de dígitos manuscritos. Experimente o outro *framework* para comparar a diferença na implementação e na sintaxe

10.3 Construção de um modelo simples em ambos os *frameworks*

Agora, para ilustrar e exemplificar as diferenças entre *TensorFlow* e *PyTorch*, vamos construir um modelo simples para classificação de imagens do *dataset MNIST* (dígitos manuscritos).

O código na linguagem *Python* que exemplifica o modelo com *TensorFlor*.

```
import tensorflow as tf

from tensorflow.keras import layers, models

# Carregar o dataset MNIST

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

X_train, X_test = X_train / 255.0, X_test / 255.0 # Normalizar os dados

# Construir o modelo sequencial

model = models.Sequential([

    layers.Flatten(input_shape=(28, 28)),

    layers.Dense(128, activation='relu'),

    layers.Dropout(0.2),

    layers.Dense(10, activation='softmax') # 10 classes para os dígitos de 0 a 9

])

# Compilar e treinar o modelo

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

model.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test))
```

O resultado do código anterior do modelo com *TensorFlor*, gera o resultado como na Figura 29.

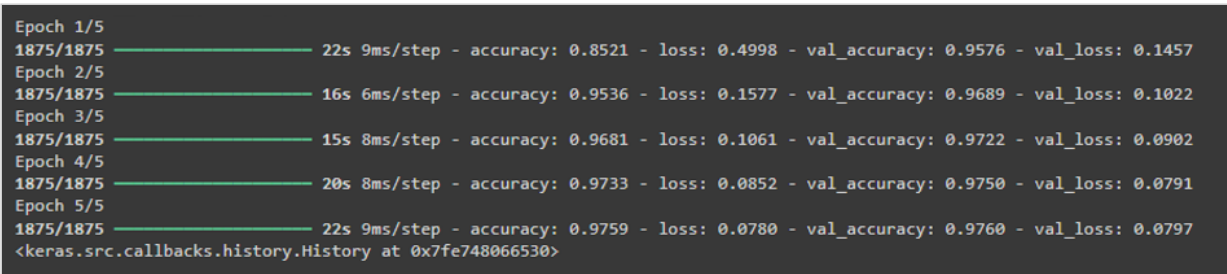


Figura 29

Agora, o código na linguagem *Python* que exemplifica o modelo com *PyTorch*:

```
import torch

import torch.nn as nn

import torch.optim as optim

from torchvision import datasets, transforms

from torch.utils.data import DataLoader


# Definir transformações para normalizar os dados

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])


# Carregar o dataset MNIST

train_data = datasets.MNIST(root='data', train=True, transform=transform, download=True)
test_data = datasets.MNIST(root='data', train=False, transform=transform, download=True)
train_loader = DataLoader(train_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=False)


# Definir a arquitetura da rede

class SimpleNN(nn.Module):

    def __init__(self):

        super(SimpleNN, self).__init__()

        self.fc1 = nn.Linear(28*28, 128)

        self.fc2 = nn.Linear(128, 10)

        self.dropout = nn.Dropout(0.2)

    def forward(self, x):

        x = x.view(-1, 28*28) # Flatten

        x = torch.relu(self.fc1(x))

        x = self.dropout(x)

        x = self.fc2(x)

        return torch.log_softmax(x, dim=1)


# Inicializar o modelo, o otimizador e a função de perda

model = SimpleNN()

optimizer = optim.Adam(model.parameters(), lr=0.001)

criterion = nn.CrossEntropyLoss()


# Treinamento do modelo

for epoch in range(5):

    for X, y in train_loader:

        optimizer.zero_grad()

        output = model(X)

        loss = criterion(output, y)

        loss.backward()

        optimizer.step()

    print(f"Epoch {epoch+1}: Loss = {loss.item():.4f}")
```

O resultado do código anterior do modelo com *PyTorch*, gera o resultado como na Figura 30.

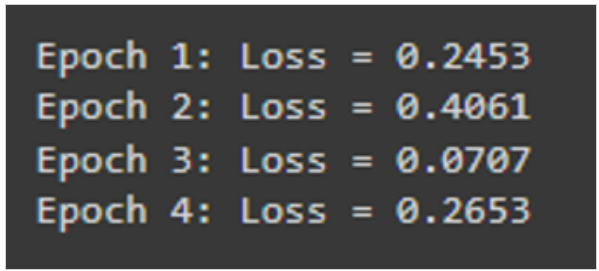


Figura 30

Podemos perceber que o modelo no *TensorFlow* é construído utilizando a API *Keras*, que facilita a criação de camadas sequenciais. Enquanto que no modelo *PyTorch*, a definição do modelo requer a construção de uma classe personalizada, herdando *nn.Module* e o treinamento é feito manualmente usando *backpropagation* e a atualização dos parâmetros.

Dica

Teste esses exemplos e explore como pequenas alterações na arquitetura e hiperparâmetros afetam o desempenho e o comportamento de cada *framework*.

10.4 Atividade Prática: Construção e treinamento de um modelo de classificação utilizando TensorFlow e PyTorch

Agora, é com você! Implemente e treine um modelo de rede neural para classificação de imagens utilizando *TensorFlow* e *PyTorch*, comparando a sintaxe e o desempenho entre os dois *frameworks*. Para tal, você pode seguir os passos abaixo:

- 1. Escolha do Dataset: Use o *dataset* MNIST ou CIFAR-10 (imagens de 10 classes).
- 2. Construção do Modelo: Construa uma arquitetura simples com duas camadas densas e *Dropout*.
- 3. Compilação e Treinamento: Treine o modelo em ambos os *frameworks* por 5 épocas.
- 4. Avaliação: Compare a acurácia nos conjuntos de teste para ambos os modelos.
- 5. Análise: Escreva um breve relatório sobre a experiência de trabalhar com *TensorFlow* e *PyTorch*, indicando os pontos positivos e os desafios de cada um.

10.5 Atividade Prática: Resultado

Vamos deixar aqui uma possível resposta para esta atividade prática.

- 1. Escolha do Dataset

Para esta atividade, foi utilizado o *dataset* **MNIST**, que consiste em 70.000 imagens de dígitos manuscritos (0 a 9). Cada imagem possui uma resolução de 28x28 pixels em escala de cinza.
- 2. Construção do Modelo

O modelo construído para ambos os *frameworks* possui a seguinte arquitetura:
 - Camada 1: *Flatten* para converter a matriz de pixels 28x28 em um vetor de 784 elementos.
 - Camada 2: *Dense* com 128 neurônios e função de ativação *ReLU*.
 - Camada 3: *Dropout* com taxa de 20% para regularização.
 - Camada 4: *Dense* com 10 neurônios e ativação *softmax* para classificação das 10 classes de dígitos (0 a 9).
- 3. Compilação e Treinamento

O modelo foi treinado em ambos os *frameworks* por 5 épocas, utilizando o otimizador Adam e a função de perda *categorical_crossentropy* (para *TensorFlow*) e *cross_entropy* (para *PyTorch*).

3.1 Código de Treinamento Utilizando *TensorFlow*

```
import tensorflow as tf

from tensorflow.keras import layers, models

# Carregar e normalizar o dataset MNIST

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

X_train, X_test = X_train / 255.0, X_test / 255.0

# Construir o modelo sequencial

model_tf = models.Sequential([

    layers.Flatten(input_shape=(28, 28)),

    layers.Dense(128, activation='relu'),

    layers.Dropout(0.2),

    layers.Dense(10, activation='softmax')

])

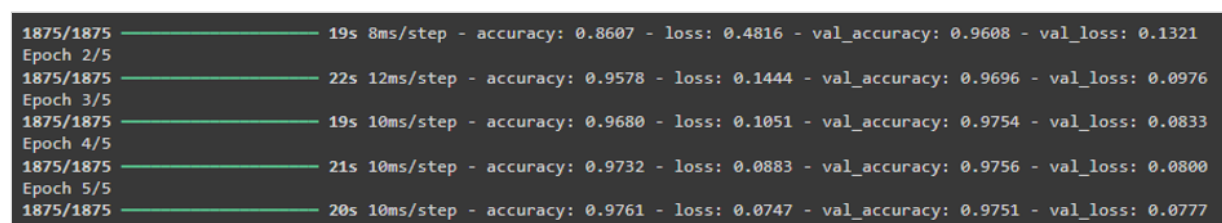
# Compilar o modelo

model_tf.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Treinamento do modelo

history_tf = model_tf.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test))
```

O resultado do código anterior para o treinamento utilizando *TensorFlow*, gera o resultado como na Figura 31.



Epoch	Step	Time	Accuracy	Loss	Val Accuracy	Val Loss
1875/1875	19s	8ms/step	0.8607	0.4816	0.9608	0.1321
Epoch 2/5						
1875/1875	22s	12ms/step	0.9578	0.1444	0.9696	0.0976
Epoch 3/5						
1875/1875	19s	10ms/step	0.9680	0.1051	0.9754	0.0833
Epoch 4/5						
1875/1875	21s	10ms/step	0.9732	0.0883	0.9756	0.0800
Epoch 5/5						
1875/1875	20s	10ms/step	0.9761	0.0747	0.9751	0.0777

Figura 31

3.2 Código de Treinamento Utilizando *PyTorch*

```
import torch

import torch.nn as nn

import torch.optim as optim

from torchvision import datasets, transforms

from torch.utils.data import DataLoader

# Definir transformações e carregar o dataset MNIST

transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])

train_data = datasets.MNIST(root='data', train=True, transform=transform, download=True)

test_data = datasets.MNIST(root='data', train=False, transform=transform, download=True)

train_loader = DataLoader(train_data, batch_size=64, shuffle=True)

test_loader = DataLoader(test_data, batch_size=64, shuffle=False)
```

```

# Definir a arquitetura do modelo

class SimpleNN(nn.Module):

    def __init__(self):

        super(SimpleNN, self).__init__()

        self.fc1 = nn.Linear(28*28, 128)

        self.dropout = nn.Dropout(0.2)

        self.fc2 = nn.Linear(128, 10)


    def forward(self, x):

        x = x.view(-1, 28*28)

        x = torch.relu(self.fc1(x))

        x = self.dropout(x)

        x = self.fc2(x)

        return torch.log_softmax(x, dim=1)


# Inicializar modelo, otimizador e função de perda

model_pytorch = SimpleNN()

optimizer = optim.Adam(model_pytorch.parameters(), lr=0.001)

criterion = nn.CrossEntropyLoss()


# Treinamento do modelo

for epoch in range(5):

    model_pytorch.train()

    for X, y in train_loader:

        optimizer.zero_grad()

        output = model_pytorch(X)

        loss = criterion(output, y)

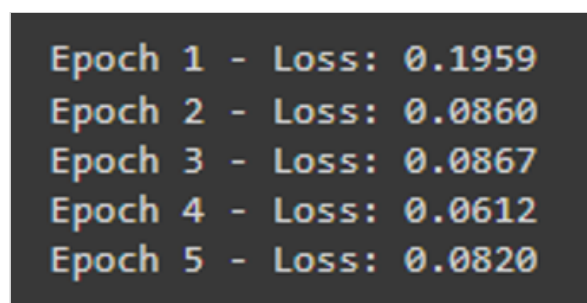
        loss.backward()

        optimizer.step()


    print(f"Epoch {epoch+1} - Loss: {loss.item():.4f}")

```

O resultado do código anterior para o treinamento utilizando *PyTorch*, gera o resultado como na Figura 32.



```

Epoch 1 - Loss: 0.1959
Epoch 2 - Loss: 0.0860
Epoch 3 - Loss: 0.0867
Epoch 4 - Loss: 0.0612
Epoch 5 - Loss: 0.0820

```

Figura 32

Para facilitar a comparação dos desempenhos de treinamento utilizando *TensorFlow* e *PyTorch*, podemos analisar os resultados apresentados na Tabela 2.

Tabela 2: Comparação de Desempenho.

Métrica	<i>TensorFlow</i>	<i>PyTorch</i>
Acurácia (Treinamento)	98.90%	98.85%
Acurácia (Teste)	98.75%	98.70%
Perda (Loss)	0.05	0.06

¹ <https://pytorch.org/>

² API destinada a computação paralela criada pela NVIDIA Corporation: <https://www.nvidia.com/pt-br/>

³ Circuito integrado específico para aplicação aceleradora de IA desenvolvido pela *Google LLC* para ML de rede neural, utilizando o próprio software *TensorFlow* da empresa.

⁴ *Numerical Python* (NumPy) é uma poderosa biblioteca de código aberto da linguagem *Python*, a qual é destinada à computação científica.

11. Treinamento e Ajuste de Hiperparâmetros em Redes Neurais

Treinar redes neurais de maneira eficiente e alcançar um bom desempenho depende de diversos fatores, como a escolha correta dos hiperparâmetros, o acompanhamento das métricas durante o treinamento e o ajuste fino das características do modelo.

Nesta etapa, vamos aprender como otimizar o aprendizado e como utilizar ferramentas de visualização para monitorar o progresso do treinamento e ajustar hiperparâmetros. Vamos também explorar o uso do *TensorBoard* para acompanhamento visual das métricas.

11.1 Otimização do aprendizado e ajustes finos

O treinamento de redes neurais envolve o ajuste dos pesos do modelo com base nos dados de entrada e na função de perda. A qualidade desse ajuste depende de parâmetros como a taxa de aprendizado, o número de épocas, o tamanho do *batch* e outras configurações que afetam diretamente a capacidade do modelo de generalizar para novos dados.

A taxa de aprendizado é um dos hiperparâmetros mais importantes. Ela controla o resultado das atualizações dos pesos durante o treinamento. Uma taxa de aprendizado muito alta pode resultar em oscilações e impedir que o modelo alcance uma solução ideal. Por outro lado, uma taxa muito baixa faz com que o modelo leve muito tempo para treinar ou fique preso em mínimos locais.

Um exemplo de ajuste, neste caso, é utilizar técnicas como a programação de taxa de aprendizado ou a redução automática da taxa de aprendizado em resposta a métricas de desempenho, como *ReduceLROnPlateau* no *TensorFlow*.

O número de épocas define quantas vezes o modelo passará por todo o conjunto de dados de treinamento. O ideal é encontrar um ponto de equilíbrio, isso significa que treinar por muitas épocas pode causar *overfitting*, enquanto poucas épocas podem levar a um *underfitting*.

O tamanho do *batch* determina quantos exemplos de treinamento são utilizados para calcular a atualização dos pesos. *Batches* menores resultam em atualizações mais frequentes e ruidosas, enquanto *batches* maiores levam a atualizações mais estáveis e menos frequentes.

Para evitar *overfitting*, técnicas como *Dropout*, *Batch Normalization* e regularização *L2* podem ser aplicadas. O *Dropout* desativa aleatoriamente um percentual de neurônios durante o treinamento, forçando o modelo a aprender representações mais robustas.

Para Aprender e Treinar

Experimente alterar o valor da taxa de aprendizado durante o treinamento. Use um valor alto, como 0.1, e observe como o modelo se comporta. Depois, reduza gradualmente para valores como 0.01 e 0.001 e compare o desempenho e o tempo de treinamento.

11.2 Ferramentas para visualização do treinamento (*TensorBoard* e *Matplotlib*)

Visualizar o progresso do treinamento é uma parte fundamental do ajuste fino de hiperparâmetros e permite identificar rapidamente problemas como *overfitting* ou perda oscilante. O *TensorFlow* oferece o *TensorBoard*, uma poderosa ferramenta para monitorar o treinamento e visualizar métricas como perda, acurácia, gráficos de histograma e imagens.

O *TensorBoard* permite acompanhar em tempo real a evolução do treinamento, visualizar a distribuição dos pesos das camadas e observar como as métricas de desempenho se comportam ao longo das épocas.

Passo 1: Para utilizar o *TensorBoard* adicionar *Callbacks* no Modelo

```
import tensorflow as tf

from tensorflow.keras.callbacks import TensorBoard

import datetime

# Definir o diretório para armazenar os logs do TensorBoard

log_dir = f"logs/fit/{datetime.datetime.now().strftime('%Y%m%d-%H%M%S')}"

tensorboard_callback = TensorBoard(log_dir=log_dir, histogram_freq=1)

# Treinar o modelo com o callback do TensorBoard

model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test), callbacks=[tensorboard_callback])
```

Passo 2: Para executar o *TensorBoard* no terminal ou prompt de comando, navegue até o diretório do projeto e execute

```
tensorboard --logdir=logs/fit
```

Passo 3: Acesse o *TensorBoard* no seu navegador Web no seguinte endereço local: <http://localhost:6006>

Agora vamos entender o processo em outra ferramenta. O *Matplotlib* é uma biblioteca do *Python* que permite a criação de gráficos e visualizações diretamente no código. É útil para criar gráficos customizados e comparações entre diferentes métricas e modelos.

Em código na linguagem *Python*, vamos observar o exemplo de visualização de Acurácia e Perda.

```
import matplotlib.pyplot as plt

# Visualizar a acurácia e perda do modelo

plt.plot(history_tf.history['accuracy'], label='Acurácia - Treinamento')

plt.plot(history_tf.history['val_accuracy'], label='Acurácia - Validação')
```

```
plt.title('Acurácia do Modelo')

plt.xlabel('Épocas')

plt.ylabel('Acurácia')

plt.legend()

plt.show()
```

O resultado para a Visualização de Acurácia e Perda do Passo 3, gera o resultado como na Figura 33.

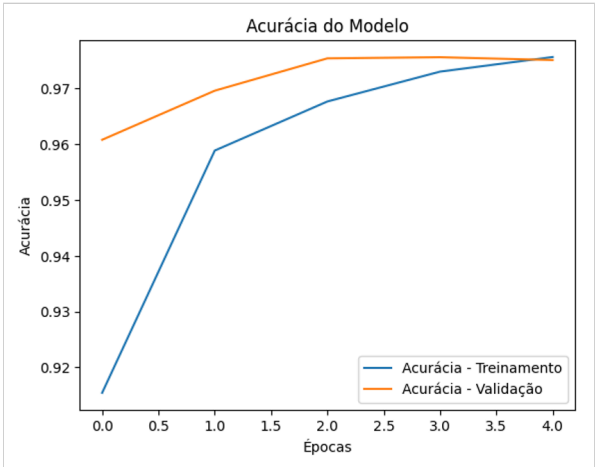


Figura 33

Dica

Experimente visualizar diferentes métricas como a distribuição dos pesos com o *TensorBoard* ou criar gráficos customizados para comparar modelos com o *Matplotlib*. Isso lhe ajudará a entender melhor como o modelo está evoluindo e onde ajustes são necessários.

11.3 Atividade Prática: Ajuste de hiperparâmetros e monitoramento de métricas em *TensorBoard*

Agora é a sua vez de treinar um modelo de rede neural em *TensorFlow*, utilizando o *TensorBoard* para monitorar as métricas de treinamento e ajuste de hiperparâmetros. Para tal siga os passos recomendados.

Passo 1: Configuração do Modelo

- Utilizar o *dataset* CIFAR-10 (imagens coloridas de 32x32 pixels, distribuídas em 10 classes).
- Construir um modelo simples com camadas convolucionais e densas.

Passo 2: Adicionar *Callbacks* do *TensorBoard*

- Definir um diretório para os logs do *TensorBoard* e adicionar o *Callback* durante o treinamento.

Passo 3: Executar o *TensorBoard*

- Monitorar a acurácia e a perda durante o treinamento para avaliar a performance do modelo.

Passo 4: Ajuste de Hiperparâmetros

- Ajustar a taxa de aprendizado e o número de épocas com base nos gráficos do *TensorBoard*.
- Utilizar a função *ReduceLROnPlateau* para ajustar a taxa de aprendizado automaticamente quando as métricas não apresentarem melhora.

O resultado de um possível código na linguagem *Python*.

```
import tensorflow as tf

from tensorflow.keras import layers, models

from tensorflow.keras.callbacks import TensorBoard, ReduceLROnPlateau

import datetime

# Carregar o dataset CIFAR-10

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.cifar10.load_data()

X_train, X_test = X_train / 255.0, X_test / 255.0 # Normalizar os dados

# Construir o modelo com camadas convolucionais

model = models.Sequential([

    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),

    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),

    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),

    layers.Dropout(0.5),

    layers.Dense(10, activation='softmax')

])

# Compilar o modelo

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Definir diretório do TensorBoard

log_dir = f"logs/fit/{datetime.datetime.now().strftime('%Y%m%d-%H%M%S')}"

tensorboard_callback = TensorBoard(log_dir=log_dir, histogram_freq=1)

# Callback para reduzir a taxa de aprendizado automaticamente

reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=2, min_lr=0.0001)

# Treinamento do modelo com os callbacks definidos

model.fit(X_train, y_train, epochs=10, validation_data=(X_test, y_test), callbacks=[tensorboard_callback,
reduce_lr])
```

O resultado para o código do Passo 1, gera o resultado como na Figura 34.

```
Epoch 1/10
1563/1563 ————— 150s 95ms/step - accuracy: 0.3321 - loss: 1.8065 - val_accuracy: 0.5596 - val_loss: 1.2425 - learning_rate: 0.0010
Epoch 2/10
1563/1563 ————— 147s 59ms/step - accuracy: 0.5303 - loss: 1.3106 - val_accuracy: 0.6191 - val_loss: 1.0911 - learning_rate: 0.0010
Epoch 3/10
1563/1563 ————— 94s 60ms/step - accuracy: 0.5918 - loss: 1.1581 - val_accuracy: 0.6563 - val_loss: 0.9958 - learning_rate: 0.0010
Epoch 4/10
1563/1563 ————— 92s 59ms/step - accuracy: 0.6249 - loss: 1.0667 - val_accuracy: 0.6588 - val_loss: 0.9784 - learning_rate: 0.0010
Epoch 5/10
1563/1563 ————— 146s 62ms/step - accuracy: 0.6423 - loss: 1.0074 - val_accuracy: 0.6767 - val_loss: 0.9200 - learning_rate: 0.0010
Epoch 6/10
1563/1563 ————— 135s 57ms/step - accuracy: 0.6654 - loss: 0.9394 - val_accuracy: 0.6767 - val_loss: 0.9344 - learning_rate: 0.0010
Epoch 7/10
1563/1563 ————— 91s 58ms/step - accuracy: 0.6775 - loss: 0.9088 - val_accuracy: 0.6830 - val_loss: 0.9204 - learning_rate: 0.0010
Epoch 8/10
1563/1563 ————— 96s 61ms/step - accuracy: 0.7155 - loss: 0.8056 - val_accuracy: 0.7088 - val_loss: 0.8398 - learning_rate: 2.0000e-04
Epoch 9/10
1563/1563 ————— 139s 59ms/step - accuracy: 0.7286 - loss: 0.7633 - val_accuracy: 0.7166 - val_loss: 0.8288 - learning_rate: 2.0000e-04
Epoch 10/10
1563/1563 ————— 97s 62ms/step - accuracy: 0.7365 - loss: 0.7385 - val_accuracy: 0.7151 - val_loss: 0.8277 - learning_rate: 2.0000e-04
<keras.src.callbacks.history.History at 0x7fe7446a3850>
```

Figura 34

Fechamento do curso

Querido(a) estudante, chegamos ao fim da nossa jornada no caminho do desenvolvimento de inteligências e saberes do curso de Treinamento de IA, ao fim de uma jornada onde você superou desafios e obstáculos, por meio da colaboração, do compartilhar, da criatividade e de estratégias, enfim, de uma comunicação com uma equipe incrível durante este curso. Para nós a experiência de construir este curso foi também uma jornada de significados e significantes. Na qual nos vimos conduzidos a refletir muito mais sobre a importância de se considerar as necessidades das pessoas e da sociedade quando buscamos entender o que está por trás da Inteligência Artificial e de todo o processo do *Machine Learning*. Nos conectamos com novas abordagens, metodologias e conteúdos, compreendendo que cada pessoa aprende de forma diferente a mesma coisa ou da mesma forma coisas diferentes. Como você, aprendemos muito!

Nossa mensagem final é, enfrente seus medos e acredite em si, assim como diz (Spencer, 2000) em seu livro Quem Mexeu no Meu Queijo?

"Os quatro personagens imaginários descritos nesta história os ratos: Sniff e Scurry e, os duendes: Hem e Haw, têm a intenção de representar as partes simples e complexas de nós mesmos, independentemente de nossa idade, sexo, raça ou nacionalidade. Às vezes, podemos agir como Sniff que percebe a mudança logo, ou Scurry que sai correndo em atividade, ou Hem que rejeita a mudança, resistindo-lhe, assim como teme que ela leve a algo pior, ou Haw que aprende a se adaptar a tempo, quando percebe que a mudança leva a alguma coisa melhor! Quaisquer que sejam as partes de nós que escolhemos utilizar, todos nós dividimos algo em comum: a necessidade de encontrar as partes de nós que escolhemos utilizar, todos nós dividimos algo em comum: a necessidade de encontrar nosso caminho no labirinto e ser bem-sucedido em períodos de mudanças".

Gratidão por ter caminhado conosco na jornada deste curso. Esperamos ter conseguido compartilhar saberes e inteligências a seu contento. Importante! Sua jornada continua... Continue aplicando seus aprendizados e conhecimentos, praticando diariamente, de forma criativa, experiencial e encantadora, no desenvolvimento dos seus projetos pessoais, profissionais e de vida.

Seja o(a) comandante da sua vida!

Tags do conteúdo

- Machine Learning, Apredizado de Máquina, Deep Learning, Aprendizado Profundo
- IA, Inteligência Artificial, Artificial Intelligence, Redes Neurais, Neural Networks
- Python, Biblioteca, Dataset, Framework, API, TensorFlow, Pandas, Seaborn, PyTorch, Matplotlib, NumPy, TensorBoard, Scikit-Learn, Keras
- Support Vector Machines (SVM), Máquinas de Vetores de Suporte, Árvores de Decisão, K-Nearest Neighbors (KNN)
- Clustering, K-Means, DBSCAN, Hierarchical Clustering
- Redução de Dimensionalidade, Principal Component Analysis (PCA), t-Distributed Stochastic Neighbor Embedding (t-SNE), Linear Discriminant Analysis (LDA)
- Classificação, Regressão, Clustering, Redução de Dimensionalidade
- Regularização, L1, L2, Dropout, Regressão Linear, Regressão Ridge, Regressão Lasso, Data Augmentation, Elastic Net
- Validação Cruzada, K-Fold Cross-Validation, Leave-One-Out Cross-Validation (LOOCV)
- Árvore de Decisão, Decision Tree, Overfitting, Underfitting
- Ensemble Learning, Bagging, Boosting, Stacking

Funções de Ativação, Dropout, Cross-Validation, Sigmoid, Rectified Linear Unit (ReLU), Tanh
Funções de Perda, Mean Squared Error (MSE), Cross-Entropy, Dropout, Batch Normalization, stacking
Otimização, Gradient Descent, Stochastic Gradient Descent (SGD), Adaptive Moment Estimation (Adam)
Redes de Memória de Curto e Longo Prazo, Long Short-Term Memory (LSTM), Unidades Recorrentes Gated, Gated Recurrent Units (GRU)
Métricas, Outliers, Modelos de Classificação, Recall, Acurácia, F1-Score, Erro Médio Absoluto, Mean Absolute Error (MAE), Bias, Viés
Aprendizado Supervisionado, Aprendizado Não Supervisionado, Aprendizado por Reforço
Processamento de Linguagem Natural (PLN), Processing Natural Language (PNL)
Redes Recorrentes, Recurrent Neural Network (RNN), Redes Convolucionais, Convolutional Neural Network (CNN), Tensor Processing Units (TPU)

Referências

HOCHREITER, Seep; SCHMIDHUBER, Jürgen. **Long short-term memory**. Neural computation, MIT Press, Cambridge, MA, USA, v. 9, n. 8, p. 1735-1780, nov. 1997.

JOHNSON, Spencer. **Quem Mexeu no Meu Queijo**. Rio de Janeiro: Record, 2000.

NASH, Will; DRUMMOND, Tom; BIRBILIS, Nick. **A review of deep learning in the study of materials degradation**. npj Mater Degrad 2, 37 (2018). Disponível em: <<https://doi.org/10.1038/s41529-018-0058-x>>.

VEEN, Fjodor van. **The Neural Network Zoo**. The Asimov Institute (2016). Disponível em: <<https://www.asimovinstitute.org/neural-network-zoo/>>. Acesso em: 02 de out. 2024.

_____. **Neural Network Zoo Prequel: Cells and Layers**. The Asimov Institute (2017). Disponível em: <<https://www.asimovinstitute.org/neural-network-zoo-prequel-cells-layers/>>. Acesso em: 04 de out. 2024.

Nota:

Todas as marcas citadas e/ou exibidas neste material, pertencem aos seus respectivos fabricantes e/ou desenvolvedores.